

Auditable Content Moderation for Encrypted File Stores

Abstract—An auditable content moderation scheme enables a file server to detect problematic content, e.g., CSAM, in encrypted files uploaded by a client(s). In such a system, the client uploads the encrypted file along with cryptographic metadata that allows the server to decrypt the file if the content matches an item in a blocklist. Otherwise, if the content is not on the blocklist, then the metadata does not reveal any information about the encrypted content. We present new designs for auditable content moderation schemes that (i) enable multiple file servers to use a common, approved blocklist that they cannot undetectably modify; (ii) offer server-client communication and client compute costs that scale sublinearly in the number of elements in the blocklist; and (iii) support exact and fuzzy matching of content. We demonstrate with benchmarks that our schemes are practical and outperform the state of the art.

1. Introduction

It is estimated that almost 50% of the world’s data resides on remote services, e.g., public clouds [7, 33]. Unfortunately, storage services are often misused to store and distribute harmful content, e.g., child-sexual-abuse-material (CSAM) [54]. In response, large storage providers like Apple, Google, and Amazon have developed their own mechanisms to identify such content [3, 4, 23]. Typically, these services use blocklists curated by trusted agencies, e.g., the National Center for Missing & Exploited Children (NCMEC) to identify problematic content. These mechanisms are reasonably effective as a deterrent and a means of detecting “easy” cases of proliferation [37, 38].¹

However, the task is more challenging with end-to-end encryption (E2EE) [36] because it precludes the provider from performing any form of content moderation on data uploaded to its servers. This has raised serious concerns among child safety groups that rely on tips to detect proliferation of CSAM [51]. To mitigate this tension, in 2021, Apple proposed a mechanism to detect harmful content in end-to-end encrypted photos uploaded to iCloud. The system, called the Apple PSI², leveraged Apple’s NeuralHash technology to generate a hash of the content. The hash is then compared privately against a blocklist curated by Apple. If there is

a match, then the iCloud server learns the decryption key. Sophisticated attackers can still circumvent detection by obfuscating their image, e.g., by “double-encrypting” it before uploading to iCloud, or tamper with the client-side software itself. Nonetheless, the mechanism provided several important safeguards: i) it allowed the service provider to perform basic content sanitization while supporting E2EE, ii) it acts as a deterrent against bulk uploads by unsophisticated clients [37, 38], and iii) it relieves the service provider of legal liabilities due to publicly hosting CSAM.

Ultimately, the scheme was not deployed due to public concerns that it could be misused for oversight and surveillance [15]. Indeed, Apple’s involvement in creating the blocklist, and the lack of a mechanism that would allow trusted parties, e.g., NCMEC, to authenticate the blocklist, is problematic [8, 10]. Second, the system was tied exclusively to NeuralHash, which produces false positives [29]; unfortunately, adapting the system to other kinds of perceptual hashes is non-trivial. While the first problem can be addressed through cryptographic techniques [45], the latter remains unaddressed to the best of our knowledge.

Thus, the central question of this paper is: *Can we design a content moderation scheme for end-to-end encrypted data stores with minimal trust assumptions that allows service providers to perform the same level of content sanitization as possible today without end-to-end encryption?*

Authenticated blocklists & distributed trust: We envision a setting where a group of independent parties (auditors) with semi-honest majority, e.g., law enforcement, and child safety groups, publish an unforgeable blocklist with content hashes. Such blocklists are not uncommon, e.g., the NCMEC have published lists of missing children and persons [21]. We expect that a threshold number of these independent auditors will behave semi-honestly, and posit that having a common blocklist curated by independent agencies, will build more trust in the system. It was also noted by prior work that the blocklist must be kept confidential against users, because it may be considered unlawful to release the information publicly, and reveal victim identities [11, 28].³

The published blocklist is used by the server(s) to identify problematic content. Matching against the blocklist can be either *exact*, where an equality test between content hashes suffices, or *fuzzy*, where a metric distance is computed between content hashes to estimate similarity against the blocklisted items [55]. The latter is important to identify

1. Of course, a motivated attacker can still store CSAM by encrypting the content before uploading; it is unclear whether any satisfactory solution can solve this problem without completely violating client-side privacy.

2. Apple’s original whitepaper is no longer available [9], however, the technical description of the scheme is available online [40, 43], and a follow up published work validates the technical claims [45].

3. We do not substantiate these claims, but treat confidentiality as a necessary feature as it is the accepted model in all prior works [9, 28, 45].

content that is obfuscated by simple transformations, e.g., image cropping or rotations.

We formalize this idea by designing *auditable content moderation schemes* that support both exact and fuzzy matching. The core building blocks of our constructions are blocklists that can be stored and authenticated by the client with reasonable storage and compute costs while ensuring: i) **confidentiality**: clients do not learn the constituent blocked items; ii) **unforgeability**: only the auditors can create/modify the blocklist; and iii) **oblivious matching**: the client can obviously match its input against items in the blocklist while revealing the result to the server, such that if the input is not on the blocklist, then the result of the match does not reveal any information. We call such designs as *authenticated & confidential blocklists* (ACBs).

Contributions: We propose three new types of ACB designs leveraging new cryptographic techniques.

- An encrypted Bloom filter based design with $\mathcal{O}(1)$ cost to perform an oblivious match against the blocklist. We propose an *authenticated OPRF* functionality extending the standard OPRF definition to an authenticated multi-party setting to create the ACB.
- A signature based design where a signed, cryptographic accumulator serves as a $\mathcal{O}(1)$ -sized ACB. We propose a new cryptographic primitive, a *randomizable set-homomorphic signature*, that allows us to directly compute on the signature to implement an oblivious match against the blocklist with $\mathcal{O}(1)$ client cost.
- An encrypted cuckoo hash table based design that supports fuzzy matching. We leverage a new cryptographic primitive, *detectable secret-sharing*, that allows a party to identify the shares of a secret when there are at least t of them in a set, but otherwise reveals nothing.

Integration & Evaluation: We leverage these ACB designs in our content moderation schemes. Our exact matching scheme supports a *fast path* that verifies and approves a majority of the safe content in 8ms when comparing against a million-sized blocklist. This is more than **60 $\times$ faster** than the state-of-the-art [28] *that operates under a more restrictive threat model with a semi-honest server*, with **50 $\times$ lower communication costs**. The full verification for blocked content takes roughly 430 ms, but since this is presumably rare, the average cost of verification with 10% of the content requiring a full check is 43ms, **12 $\times$ faster** than the state of the art. Fuzzy matching against a blocklist with 1 million items takes less than 600ms, **46 $\times$ faster** than the state-of-the-art [28] with **25 $\times$ lower communication costs**.

Limitations & risks of content moderation: The primary motivation of this work is to develop new cryptographic mechanisms for content moderation, but we also acknowledge the limitations and risks of deploying such a system. First, as with any content moderation system, there are risks of misuse for unauthorized surveillance. Our solution tries to address this problem by distributing trust among multiple independent agencies, overwhelmingly the most effective counter-balance to pressure and compromise, and is widely used in both research and in practice (e.g., Tor).

However, we acknowledge that this may not suffice against a powerful adversary, e.g., a nation state. Thus, further discussions and potential mitigations are necessary before any content moderation system, including ours, is widely deployed. These discussions are out of scope of this paper.

Second, any system relying only on server-side checks (as in our work) is limited to detecting content that is not obfuscated by the client before uploading e.g., by locally encrypting the data. Evidence shows that even this basic level of monitoring is important for slowing down proliferation of CSAM [51], and is implemented by most service providers today. Thus, our technical solution only extends these monitoring capabilities while also allowing the service provider to support end-to-end encryption.

2. Related Work

In this work, we are interested in content moderation schemes for encrypted file stores that use blocklists. Content moderation has been studied generally for network communication and encrypted traffic streams, but these solutions cannot be immediately applied to file stores without incurring high costs or leaking matching patterns [34, 35, 48, 53]. We refer the interested reader to an excellent survey [44].

Table 1 compares the state-of-the-art schemes with our work. We exclude preprocessing and blocklist creation costs for all the schemes since they are linear in the size of the blocklist. Instead we focus on the online costs during file uploads. Auditability refers to whether the scheme allows a set of independent auditors to create and verify the blocklist. Our protocols support a *fast path* for verification assuming that most of the uploaded content will not be blocked; we present the average case and worst case costs.

Kulshrestha and Mayer [28] introduced a system with a blocklist of perceptual hashes of CSAM content. The protocol leverages a single-server PIR scheme, and supports both exact and approximate matching against the blocklist. *The protocol only supports a semi-honest service provider, and cannot be trivially extended to provide malicious security without incurring large client costs, e.g., by using an authenticated PIR* [16]. The scheme also does not allow independent auditors to verify the blocklist.

Apple developed a mechanism to detect CSAM content in encrypted files uploaded to iCloud [4] using the NeuralHash model [9]. A core criticism of this work is that the blocklist is created solely by Apple. This criticism was addressed by Scheffler et al. [45] by leveraging an aggregated signature scheme. *Nonetheless, the mechanism does not generalize to other types of content hashes, including ones that require fuzzy matching*. This is a drawback because NeuralHash without fuzzy matching is easy to circumvent, and also suffers from false positives [29, 50].

Gap in research. We observe from Table 1 that there is no scheme that provides auditability, supports both exact and fuzzy matching, and protects against a malicious server, while also ensuring that the client compute and communication costs remain sublinear in the blocklist size. Our work addresses these requirements.

Table 1: Encrypted content moderation systems.

n : blocklist size, $\epsilon \in (0, 1/2)$, ℓ : vector length, SH: semi-honest, M: malicious, Avg: average-case cost, WC: worst-case cost

Type of Matching	System Name	Audibility	Threat model		Asymptotic Costs			
			Server	Client	Client Comp.	Comm. Volume	Client Storage	Server Comp.
Exact	Private Hash Matching [28]	N/A	SH	SH	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(1)$	$\mathcal{O}(n)$
	Apple PSI [9] + Agg. Signature [45]	✓	M	SH	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
	This work	✓	M	SH	Avg: $\mathcal{O}(1)$ WC: $\mathcal{O}(\log n)$	$\mathcal{O}(n^\epsilon)$	$\mathcal{O}(n^\epsilon)$	Avg: $\mathcal{O}(1)$ WC: $\mathcal{O}(n)$
Fuzzy	Private Hash Matching [28]	N/A	SH	SH	$\mathcal{O}(\sqrt{n\ell})$	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(1)$	$\mathcal{O}(n\ell)$
	This work	✓	M	SH	$\mathcal{O}(n^\epsilon)$	$\mathcal{O}(n^\epsilon)$	$\mathcal{O}(n^{1+\epsilon})$	$\mathcal{O}(n^{3\epsilon})$

3. Model

A content moderation scheme enables a file server to detect harmful content, e.g., CSAM, in encrypted files uploaded by a client(s) by matching file metadata against items on a *blocklist*. If the content is not on the blocklist, then the metadata does not reveal any information. Typically, a blocklist is created from a content hash [9] or an *embedding* [19, 30, 42]. The blocklist is published by a group of *auditors*. Clients can verify its authenticity using a public cryptographic token, e.g., a signature. In addition, clients do not learn any information about the blocklisted items.

Depending on the type of items, content matching may either be in the form of *exact matching* or *fuzzy matching*. Exact matching is an equality test between an input and the items on the blocklist, while fuzzy matching computes a distance between content embeddings and compares the distance against a threshold. In both cases, the goal is to detect content that is similar (and not just identical) because simple transformations like adding a filter will make the content different enough from a nearby blocklisted item. Typically, exact matching is performed over learned content hashes, e.g., a NeuralHash [4] that derives identical strings or integers (with high probability) from similar content. Fuzzy matching leverages an embedding function that maps content x_1 and x_2 to corresponding elements in a metric space $\mathcal{M}$ with distance $\text{dist}(\cdot, \cdot)$. If the content is similar then $\text{dist}(\text{embed}(x_1), \text{embed}(x_2)) \leq t$ with high probability, where t is a threshold, and the probability is taken over random selection of the embedding function.

3.1. Threat Model

Our content moderation system has three parties.

- 1) **Auditors:** There is a group of agencies that publish and/or verify the items on the blocklist. The auditors may include representatives from law enforcement, social rights groups, independent journalists, etc. Some of the auditors may also be untrusted, i.e., they may collude with the file servers. For this work, we assume that a majority of the auditors are honest⁴.
- 2) **File server:** The server has the blocklist, although it may not necessarily participate in creating it. The file server may behave maliciously to breach confidentiality, but it ensures integrity and availability.

4. The honest majority assumption is an artifact of the cryptographic tool we use but it is not a fundamental requirement of the framework.

- 3) **Client:** The client is semi-honest, i.e., it will follow protocols. While it might appear that the assumption is too strong, this model has largely proven successful in detecting content like CSAM and acting as a deterrent against less sophisticated users [9, 28, 45]. Moreover, sophisticated users capable of circumventing checks, e.g., by modifying the client-side code, can also use unmonitored channels for storing harmful content.

On the semi-honest client assumption: Before proceeding further with our security goals, we pause to consider the practicality of our security model. Of course, the obvious objection is that clients who upload CSAM may not follow protocols since circumventing detection is ultimately their goal. Indeed, a malicious client can leverage tools for adversarial noising [47], or “double-encrypt” their files so that the client-side software only observes random data. Unfortunately, it is unclear how such attacks can be prevented without using more sophisticated cryptographic tools such as general purpose verifiable computation [20], or trusted execution environments [25]. Both approaches add overheads/assumptions that are not suitable for resource-constrained client devices. Consequently, **all encrypted content moderation tools for detecting CSAM, including Apple’s (proposed but now defunct) solution to detect CSAM on iCloud, assume a semi-honest client.**

Moreover, there are several reasons to consider semi-honest clients a realistic threat. First, there is ample evidence that CSAM is often uploaded by unsophisticated users “in the clear” [37, 38]. However, if cloud providers support end-to-end encryption by default (as it is emerging as a de-facto standard), the opportunities for detecting even such obvious cases are hindered. A tool for a semi-honest client will enable servers to at least perform a baseline level of detection. Second, our mechanisms are compatible with content embeddings that are resilient against adversarial manipulations [55] by the client while *also tolerating a malicious server*. Existing work does not simultaneously consider both of these potential threats (Table 1), and therefore our work is an important first step towards designing schemes for a more realistic threat model.

3.2. Security Goals & Preliminaries

Our security goal is more conservative than existing work [28, 45], and the proposed Apple PSI system. In particular, unlike existing systems, which allow the server to

$\mathcal{F}_{\text{ExactMod}}$: Auditable Content Moderation with Exact Matching Parameters: η auditors $A_1, \dots, A_\eta$, a server SERVER and a client CLIENT. A signature scheme $\Pi_{\text{Sig}} = \langle \text{SGEN}, \text{SSIGN}, \text{SVRFY} \rangle$. False negative rate $\epsilon_{\text{fnr}} \in (0, 1)$ and false positive rate $\epsilon_{\text{fpr}} \in (0, 1)$. Creating blocklist: 1) Upon receiving the message $(\text{CreateBlocklist}, A_k, \mathcal{B}, sk_k)$ from A_k, store the blocklist and the auditor's private key 2) After receiving the CreateBlocklist message from all auditors, abort if the blocklists submitted by the auditors are not the same. Otherwise, output $\mathcal{T}_{\mathcal{B}}$ created from $\mathcal{B}$, and signatures under the auditor's keys $\{\text{SSIGN}_{sk_k}(\mathcal{T}_{\mathcal{B}})\}_{k \in [1, \eta]}$ to CLIENT. Output the blocklist $\mathcal{B}$ to SERVER and store $\mathcal{B}$ Verifying uploads: 3) Initialize a list RetList 4) Upon receiving $(\text{UPLOADFILE}, \text{ID}, C_{\text{DATA}})$ from CLIENT, store $\langle \text{ID}, C_{\text{DATA}} \rangle$ in RetList 5) Upon receiving $(\text{CHECKFILE}, \text{ID}, C'_{\text{DATA}})$ from SERVER, a) If there is no entry $\langle \text{ID}, _ \rangle$ in KeyList, then abort. b) If there is an entry $\langle \text{ID}, C'_{\text{DATA}} \rangle \in \text{RetList}$, and $C'_{\text{DATA}} \in \mathcal{B}$, then return 1 with probability at least $1 - \epsilon_{\text{fnr}}$. c) If there is an entry $\langle \text{ID}, C'_{\text{DATA}} \rangle \in \text{RetList}$, and $C'_{\text{DATA}} \notin \mathcal{B}$, then return 0 with probability at least $1 - \epsilon_{\text{fpr}}$.	$\mathcal{F}_{\text{FuzzyMod}}$: Auditable Content Moderation with Fuzzy Matching Parameters: η auditors $A_1, \dots, A_\eta$, a server SERVER and a client CLIENT. A signature algorithm $\Pi_{\text{Sig}} = \langle \text{SGEN}, \text{SSIGN}, \text{SVRFY} \rangle$. Threshold t for a metric distance $\text{dist}(\cdot, \cdot)$. A false positive rate $\epsilon_{\text{fpr}} \in (0, 1)$ and false negative rate $\epsilon_{\text{fnr}} \in (0, 1)$. Creating blocklist: 1) Upon receiving the message $(\text{CreateBlocklist}, A_k, \mathcal{B}, sk_k)$ from auditor A_k, store the blocklist and the auditor's private key 2) After receiving the CreateBlocklist message from all auditors, abort if the blocklists submitted by the auditors are not the same. Otherwise, output $\mathcal{T}_{\mathcal{B}}$ created from $\mathcal{B}$, and signatures under the auditor's keys $\{\text{SSIGN}_{sk_k}(\mathcal{T}_{\mathcal{B}})\}_{k \in [1, \eta]}$ to CLIENT. Output the blocklist $\mathcal{B}$ to SERVER and store $\mathcal{B}$ Verifying uploads: 3) Initialize a list RetList 4) Upon receiving $(\text{UPLOADFILE}, \text{ID}, C_{\text{DATA}})$ from CLIENT, store $\langle \text{ID}, C_{\text{DATA}} \rangle$ in RetList 5) Upon receiving $(\text{CHECKFILEFUZZY}, \text{ID}, C'_{\text{DATA}})$ from SERVER, a) If there is no entry $\langle \text{ID}, _ \rangle$ in RetList, then abort. b) If there is an entry $\langle \text{ID}, C'_{\text{DATA}} \rangle \in \text{RetList}$ • Check if there is some entry $b \in \mathcal{B}$ such that $\text{dist}(b, C'_{\text{DATA}}) \leq t$. If so, return 1 with probability $\geq 1 - \epsilon_{\text{fnr}}$. Otherwise return 0. • If for all $b \in \mathcal{B}$, $\text{dist}(b, C'_{\text{DATA}}) > t$, return 0 with probability $\geq 1 - \epsilon_{\text{fpr}}$. Otherwise return 1.
---	--

Figure 1: Ideal functionality for auditable content moderation with exact and fuzzy matching

obtain a decryption key when the client's content matches some item on the blocklist, *we do not divulge the encrypted content to the server under any circumstance*. Instead, the server is only able to identify potentially problematic content by matching a content hash or embedding against the blocklist, and then decide whether to accept the encrypted content. If the content hash does not match a blocklisted item, the server does not learn any further information. The server may suspend or report an account if it observes multiple uploads from the same client that are deemed problematic. Our mechanism is agnostic to this decision.

Ideal functionality: We define the ideal functionality of an auditable content moderation scheme in Fig. 1. $\mathcal{F}_{\text{ExactMod}}$ supports exact matching against the blocklist, while $\mathcal{F}_{\text{FuzzyMod}}$ supports a fuzzy match based on a metric distance. In the functionality, the auditors invoke CreateBlocklist to create a blocklist from a set of items $\mathcal{B}$. The functionality outputs an authenticated and confidential blocklist $\mathcal{T}_{\mathcal{B}}$ (defined later), and signs $\mathcal{T}_{\mathcal{B}}$ under the private keys of all the auditors. The client downloads and stores $\mathcal{T}_{\mathcal{B}}$ after verifying the signatures of all the auditors.

During an upload, the client invokes the functionality with the message UPLOADFILE providing an input C_{DATA} that is derived from both the file content it wants to upload and $\mathcal{T}_{\mathcal{B}}$. The functionality stores C_{DATA} along with a file identifier. C_{DATA} can be either a hash of the file or a metric space embedding, depending on the type of matching. After the client uploads the file along with C_{DATA} , the server checks whether $C_{\text{DATA}} \in \mathcal{B}$ by invoking the functionality with the CHECKFILE (resp. CHECKFILEFUZZY in $\mathcal{F}_{\text{FuzzyMod}}$) message. The functionality returns the result with a fixed false positive rate ϵ_{fpr} and false negative rate ϵ_{fnr} .

The false positive rate and false negative rate is included because content/image embeddings are not perfect and typically produce false positives and false negatives. In the context of content moderation, false positives are detrimental to privacy. Thus, as we will describe, our protocol realizing $\mathcal{F}_{\text{ExactMod}}$ has no false positives. In contrast, the protocol realizing $\mathcal{F}_{\text{FuzzyMod}}$ has a tunable false positive rate and false negative rate, and does not reveal any additional information, including the matching blocklisted item, when there is a false positive – the server only learns a 0/1 result.

We remark that the functionality allows the client to upload multiple files with the same C_{DATA} . In practice, a client may want to avoid leaking such correlations to the server. We overcome this leakage in our protocols. Finally, while blocklisted items are protected from a client that performs an *offline* dictionary attack using $\mathcal{T}_{\mathcal{B}}$, we do not consider an *online* dictionary attack a realistic threat – where a client probes the blocklist by intentionally uploading blocked content and observes the results of the match – because once multiple uploaded items are deemed problematic, presumably the account will be suspended.

Notation: We use $\leftarrow$ to denote assignment of the value on its right to the variable on its left, and $\leftarrow \$$ to sample uniformly from the set on its right and assign the sampled value to the variable on its left. Let $g : \mathcal{K} \times \mathcal{D} \rightarrow \mathcal{R}$ denote a function family with *keyspace* $\mathcal{K}$, *domain* $\mathcal{D}$, and *range* $\mathcal{R}$. For $k \in \mathcal{K}$, we denote by $g_k : \mathcal{D} \rightarrow \mathcal{R}$ the function with $g_k(d) = g(k, d)$ for each $d \in \mathcal{D}$. λ denotes a security parameter.

Cryptographic tools: We use standard cryptographic tools: i) an IND-CPA secure symmetric key encryption scheme $\Pi_{\text{enc}} = \langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle$, ii) a digital signature scheme $\Pi_{\text{Sig}} = \langle \text{SGEN}, \text{SSIGN}, \text{SVRFY} \rangle$, and iii) an IND-CPA se-

cure additively-homomorphic encryption scheme $\Pi_{\text{ahe}} = \langle \text{AH_KEYGEN}, \text{AH_ENC}, \text{AH_DEC} \rangle$. The encryption scheme is homomorphic under the group operation. That is, there is an efficient algorithm to compute a ciphertext $\text{Enc}_{pk_e}(m_1 + m_2)$ from pk , $c_1 = \text{Enc}_{pk_e}(m_1)$, and $c_2 = \text{Enc}_{pk_e}(m_2)$; we denote this operation by $c_1 +_{pk} c_2$.

We will also make use of a pseudorandom function family, a pseudorandom permutation family and a pseudorandom number generator; the specifications are provided in the sections they are used. We refer to a textbook on cryptography, e.g., [27] for formal definitions.

4. Blocklist Constructions

The core building block we leverage to realize the ideal functionalities described in Sec. 3.2 are authenticated and confidential blocklists (ACB), jointly created by the auditors from a common set of blocklisted items.

We formally define the interfaces exposed by an ACB

- **Publish:** This is an interactive mechanism invoked by the auditors to create an ACB from a common set of items $\mathcal{B} \leftarrow \{b_1, \dots, b_n\} \subseteq \mathcal{U}$ and publish the output $\mathcal{B}^{(\text{blind})}$. An ACB ensures

- *Confidentiality:* for any PPT adversary $\mathcal{A}$

$$\mathbb{P}(b \leftarrow \mathcal{A}(\mathcal{B}^{(\text{blind})}) \wedge b \in \mathcal{B}) \leq 1/|\mathcal{U}| + \text{negl}(\lambda)$$

- *Unforgeability:* The auditors sign $\mathcal{B}^{(\text{blind})}$ (and other relevant information) under their own keys.

- **Preprocess:** This is an optional interactive mechanism between the client and the server where they can process the published blocklist to create a representation $\mathcal{T}_{\mathcal{B}}$. $\mathcal{T}_{\mathcal{B}}$ is stored on the client and supports oblivious matching against $\mathcal{B}$ during uploads. The preprocessing may require secrets from the client and the server. We also support constructions where preprocessing is not necessary to support matching. In that case, $\mathcal{T}_{\mathcal{B}} = \mathcal{B}^{(\text{blind})}$.
- **Search:** This is an interactive mechanism invoked by the client and the server where given an input y from the client, the mechanism outputs either TRUE or FALSE to the server depending on whether $y \in \mathcal{B}$, and a predefined false positive rate and a false negative rate⁵.

$$\mathbb{P}(\text{Search}(y, \mathcal{T}_{\mathcal{B}}) = \text{TRUE} \mid y \in \mathcal{B}) \geq 1 - \epsilon_{\text{fnr}}$$

$$\mathbb{P}(\text{Search}(y, \mathcal{T}_{\mathcal{B}}) = \text{FALSE} \mid y \notin \mathcal{B}) \leq 1 - \epsilon_{\text{fpr}}$$

where $\epsilon_{\text{fnr}} \in (0, 1)$ is the false negative rate, and $\epsilon_{\text{fpr}} \in (0, 1)$ is the false positive rate. The probability is over the choice of random coins used in creating $\mathcal{T}_{\mathcal{B}}$ from $\mathcal{B}$.

- **TSearch:** This is an optional interactive mechanism invoked by the client and server where given a set of inputs

5. While the mechanism only returns a TRUE/FALSE, the protocols may additionally reveal y when $y \in \mathcal{B}$. This leakage is inevitable since there is no lower bound on the size of $\mathcal{B}$, and as such, the server can easily guess y when there is a match against a small blocklist.

$Y \leftarrow \{y_1, \dots, y_\ell\}$ from the client, and a public threshold t , the mechanism outputs a value in $\{\text{TRUE}, \text{FALSE}\}$.

$$\mathbb{P}(\text{TSearch}(Y, \mathcal{T}_{\mathcal{B}}) = \text{TRUE} \mid |Y \cap \mathcal{B}| \geq t) \geq 1 - \epsilon_{\text{fnr}}$$

$$\mathbb{P}(\text{TSearch}(Y, \mathcal{T}_{\mathcal{B}}) = \text{FALSE} \mid |Y \cap \mathcal{B}| < t) \geq 1 - \epsilon_{\text{fpr}}$$

where $\epsilon_{\text{fnr}} \in (0, 1)$ is the false negative rate, and $\epsilon_{\text{fpr}} \in (0, 1)$ is the false positive rate. The probability is taken over the choice of the random coins used in creating $\mathcal{T}_{\mathcal{B}}$ from $\mathcal{B}$. For Search and TSearch, if the client's input(s) is not on the blocklist, then no additional information about y (and Y resp.) is revealed to the server.

We will introduce three different types of authenticated blocklists, and use these constructions later in Sec. 5 to design our content moderation schemes.

4.1. Bloom Filter Based Blocklist

A Bloom filter provides a succinct representation of a set with storage costs that are sublinear in the set size. A Bloom filter is ideally suited for a large blocklist since it is stored on the client. However, a Bloom filter does not provide confidentiality of the blocklisted items⁶. We introduce confidentiality by constructing the Bloom filter with obfuscated items, while developing new mechanisms to support efficient and oblivious search. This requires a new cryptographic primitive called an *authenticated OPRF*.

4.1.1. Authenticated OPRF. We define a functionality called *authenticated OPRF*, that extends the definition of an oblivious pseudorandom function (OPRF) to a functionality $\mathcal{F}_{\text{authOPRF}}$ (Fig. 3), where a group of η auditors with semi-honest majority jointly sign a list of inputs to a pseudorandom function. $\mathcal{F}_{\text{authOPRF}}$ receives the key for the PRF from the client acting as the sender. The receiver of the PRF evaluations is the server, who does not provide inputs.

Realizing $\mathcal{F}_{\text{authOPRF}}$: We realize $\mathcal{F}_{\text{authOPRF}}$ by leveraging the 2HashDH PRF construction [24] in a group $\mathcal{G}$ where the OM gap-DH problem is hard. The construction is $\text{PRF}_{\kappa_{\text{PRF}}}^{\mathcal{C}}(x) = H_2(x, H_1(x)^{\kappa_{\text{PRF}}^{\mathcal{C}}})$, where $H_1 : \mathcal{U} \rightarrow \mathcal{G}$ and $H_2 : \{0, 1\}^{|\mathcal{U}|+|\mathcal{G}|} \rightarrow \{0, 1\}^\lambda$ are random oracles. The protocol is in Fig. 6 in the Appendix. In brief,

- 1) The auditors “blind” the blocklist elements and publish an ordered set $\mathcal{B}^{(\text{blind})} \leftarrow \{\langle i, H_1(b_i) \rangle^{r_i}\}_{b_i \in \mathcal{B}}$. The server is provided the blinding factors and the (ordered) blocklist elements $\langle 1, b_1, r_1 \rangle, \dots, \langle n, b_n, r_n \rangle$.
- 2) The auditors sign (a hash of) $\mathcal{B}^{(\text{blind})}$ under their private keys. The client obtains $\mathcal{B}^{(\text{blind})}$ and the signatures.
- 3) After verifying the signatures, the client exponentiates the blinded elements in $\mathcal{B}^{(\text{blind})}$ with its PRF key $\kappa_{\text{PRF}}^{\mathcal{C}}$ and produces another ordered set $\{\langle i, (H_1(b_i)^{r_i})^{\kappa_{\text{PRF}}^{\mathcal{C}}} \rangle\}_{b_i \in \mathcal{B}}$ which is sent to the server.

6. A Bloom filter without confidentiality was considered for content moderation in prior work [28] but assumed to be unsatisfactory.

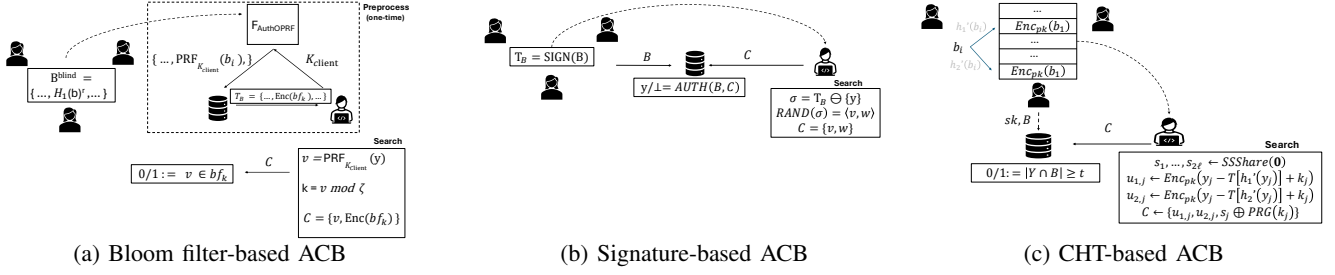


Figure 2: Authenticated and confidential blocklist constructions

$\mathcal{F}_{\text{authOPRF}}$: Ideal Functionality for Authenticated OPRF

Parameters: η auditors $A_1, \dots, A_\eta$, a sender CLIENT and a receiver SERVER. A pseudorandom function $\text{PRF} : \mathcal{K}_{\text{PRF}} \times \mathcal{U} \rightarrow \{0, 1\}^\lambda$, a signature scheme $\Pi_{\text{Sig}} = \langle \text{SGEN}, \text{SSIGN}, \text{SVRFY} \rangle$, a hash function $h : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$

Inputs: The auditors have a common list of item $\mathcal{B} \subseteq \mathcal{U}$ where $|\mathcal{B}| = n$; auditor A_k has public-private key pair $\langle pk_k, sk_k \rangle_{k \in [\eta]}$ for Π_{Sig} . CLIENT has the key κ_{PRF} for PRF. SERVER has no input.

Functionality:

- Upon receiving the message (Publish, $A_k, \mathcal{B}, sk_k$) from A_k , store $\mathcal{B}$ and sk_k .
- After receiving the Publish message from all auditors, abort if $\mathcal{B}$ submitted by the auditors are not the same. Otherwise,
 - Initialize a key-value store KV.
 - For each $i \in [1, n]$ and $b_i \in \mathcal{B}$, add $\langle i, b_i, r_i \rangle$ to KV, where $r_i \leftarrow \{0, 1\}^\lambda$.
 - For each $\langle i, b_i, r_i \rangle \in \text{KV}$, add $\langle i, r_i \rangle$ to a set $\mathcal{B}^{(\text{blind})}$.
 - Sign hash of $\mathcal{B}^{(\text{blind})}$ under the auditors' private keys $\Sigma_k \leftarrow \text{SSIGN}_{sk_k}(h(\mathcal{B}^{(\text{blind})}))$.
 - Output $\mathcal{B}^{(\text{blind})}$ and $\{\Sigma_k\}_{k \in [1, \eta]}$ to CLIENT. Output KV to SERVER.
- Upon receiving the message (Eval, $\mathcal{B}^{(\text{blind})}, \kappa_{\text{PRF}}$) from CLIENT, for each $\langle i, b_i, r_i \rangle \in \text{KV}$, output to SERVER $\langle b_i, \text{PRF}_{\kappa_{\text{PRF}}}(b_i) \rangle$.

Figure 3: $\mathcal{F}_{\text{authOPRF}}$: Authenticated OPRF

- 4) The server removes the blinding by computing $\left((H_1(b_i)^{r_i})^{\kappa_{\text{PRF}}^C} \right)^{r_i^{-1}}$ for each $\{ \langle i, (H_1(b_i)^{r_i})^{\kappa_{\text{PRF}}^C} \rangle \}_{b_i \in \mathcal{B}}$.

Thus, the server obtains $\{ \langle b_i, H_2(b_i, H_1(b_i)^{\kappa_{\text{PRF}}^C}) \rangle \}_{b_i \in \mathcal{B}} = \{ \langle b_i, \text{PRF}_{\kappa_{\text{PRF}}^C}(b_i) \rangle \}_{b_i \in \mathcal{B}}$.

4.1.2. Blocklist construction. We create a Bloom-filter-based ACB $\mathcal{T}_{\text{BF}}$ using $\mathcal{F}_{\text{authOPRF}}$ as shown in Algorithm 1 and depicted in Fig. 2a. At a high level, the auditors invoke $\mathcal{F}_{\text{authOPRF}}$ with Publish to publish a blinded blocklist $\mathcal{B}^{(\text{blind})}$, while the client and server invoke Eval with $\mathcal{B}^{(\text{blind})}$ as input (line 6). Using the output $\{ \langle b_i, \text{PRF}_{\kappa_{\text{PRF}}^C}(b_i) \rangle \}_{b_i \in \mathcal{B}}$ from $\mathcal{F}_{\text{authOPRF}}$, the server creates $\zeta \leftarrow \lceil n / \log n \rceil$ Bloom filters, $bf_1, \dots, bf_\zeta$, where $n = |\mathcal{B}|$. The items b and $\text{PRF}_{\kappa_{\text{PRF}}^C}(b)$ are added to bf_k where $k = \text{PRF}_{\kappa_{\text{PRF}}^C}(b) \bmod \zeta$ (lines 8 - 15). Finally, the Bloom filters are encrypted by the server under a symmetric key and stored by the client.

We remark that instead of directly storing $\text{PRF}_{\kappa_{\text{PRF}}^C}(b)$ in a Bloom filter, the server stores $\text{PRP}_{\kappa_{\text{PRP}}^S}(\text{PRF}_{\kappa_{\text{PRF}}^C}(b))$, where PRP is a pseudorandom permutation and κ_{PRP}^S is a

key sampled by the server. This is because the client will be informed when there is a match in our content moderation scheme (described later in Sec. 5). However, a match can also result from a false positive which leaks information to the client about the other items stored in the Bloom filter. This in turn leaks information about items in the blocklist; note that the items in the Bloom filter are obfuscated under the client's PRF key, and as such the client can "reverse-engineer" blocklist values when there is a collision. The PRP ensures that the collisions during a false positive are between items that are indistinguishable from random, and as such the occurrence of a false positive can be *simulated* against a semi-honest client in our security proof.

During a search with input y , the client sends to the server $z \leftarrow \text{PRF}_{\kappa_{\text{PRF}}^C}(y)$ and the encrypted Bloom filter bf_k where $k = z \bmod \zeta$. The server checks if $\text{PRP}_{\kappa_{\text{PRP}}^S}(z) \in bf_k$, and if so, the search returns a positive (lines 21 - 22). To prevent the server from correlating searches, the client maintains a list of previously checked (and approved) items, $\mathcal{L}$. If $y \in \mathcal{L}$, then the client sets $z \leftarrow \{0, 1\}^\lambda$.

Costs: The costs of creating and searching are as follows.

- CreateBlocklist has $\mathcal{O}(n)$ cost for the auditors, client and server. This is a one-time cost.
- Search has $\mathcal{O}(1)$ cost for the client and the server, and $\mathcal{O}(\log n)$ communication cost for a single Bloom filter.
- The total storage cost for the client is $\sum_{k \in [1, \zeta]} |bf_k|$.

Security: Confidentiality and unforgeability of $\mathcal{T}_{\text{BF}}$ is ensured by $\mathcal{F}_{\text{authOPRF}}$, and IND-CPA security. During a search, the server learns $\text{PRF}_{\kappa_{\text{PRF}}^C}(y)$, which is indistinguishable from random to the server if $y \notin \mathcal{B}$. Recall that the server only learns $\text{PRF}_{\kappa_{\text{PRF}}^C}(b)$ for $b \in \mathcal{B}$. A full proof is omitted.

4.2. Homomorphic Signature Based Blocklist

There are two drawbacks of the Bloom filter based ACB: i) the storage overhead for $\mathcal{T}_{\text{B}}$ is $\mathcal{O}(n)$, and ii) the search can produce false positives. In this section, we will design an ACB that mitigates both of these problems, but with higher computational costs for the server during search. The construction leverages a new cryptographic primitive called a *randomizable set homomorphic signature*. Intuitively, the ACB is a signature on the accumulated set of blocklisted items [26] under the auditors' private keys. The signature

Algorithm 1 Π_{bf} : Bloom Filter Based Blocklist

Require: A signature scheme $\Pi_{\text{Sig}} = \langle \text{SGEN}, \text{SSIGN}, \text{SVRFY} \rangle$; a pseudo-random permutation $\text{PRP} : \mathcal{K}_{\text{PRP}} \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$; a collision-resistant hash function $\{0, 1\}^* \rightarrow \{0, 1\}^\lambda$; a pseudorandom function family $\text{PRF} : \mathcal{K}_{\text{PRF}} \times \mathcal{U} \rightarrow \{0, 1\}^\lambda$; a symmetric key encryption scheme $\Pi_{\text{enc}} = \langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle$

```

1: procedure Publish( $\mathcal{B}$ )
2:   Auditor  $A_k$  samples a key pair  $\langle pk_k, sk_k \rangle$  for  $\Pi_{\text{Sig}}$ 
3:   Given blocklisted items  $\mathcal{B} \subseteq \mathcal{U}$ , the auditors invoke  $\mathcal{F}_{\text{authOPRF}}$ 
   with the Publish message with  $\mathcal{B}$  and their private key  $sk_k$  as the
   input. This step outputs  $\mathcal{B}^{(\text{blind})}$ ,  $\{\text{SSIGN}_{sk_k}(h(\mathcal{B}^{(\text{blind})}))\}_{k \in [1, \eta]}$ 

4: procedure Preprocess( $\mathcal{B}^{(\text{blind})}$ )
5:   CLIENT downloads  $\mathcal{B}^{(\text{blind})}$ ,  $\{\text{SSIGN}_{sk_k}(h(\mathcal{B}^{(\text{blind})}))\}_{k \in [1, \eta]}$  and
   verifies the signature. Then, it invokes  $\mathcal{F}_{\text{authOPRF}}$  with the Eval
   message with  $\mathcal{B}^{(\text{blind})}$  and  $\kappa_{\text{PRF}} \leftarrow \mathcal{K}_{\text{PRF}}$  as inputs.
6:   CLIENT initializes an empty set  $\mathcal{L} \leftarrow \emptyset$ 

7:   ▷ Upon receiving  $\{\langle b, \text{PRF}_{\kappa_{\text{PRF}}}^C(b) \rangle\}_{b \in \mathcal{B}}$  from  $\mathcal{F}_{\text{authOPRF}}$ ,
   SERVER implements the following
8:   Initialize  $\zeta = \lceil n / \log n \rceil$  Bloom filters  $bf_1, \dots, bf_\zeta$ , each with
   false positive rate  $\epsilon_{bf}$ 
9:   Sample  $\kappa_{\text{PRP}}^S \leftarrow \mathcal{K}_{\text{PRP}}$  and  $\kappa \leftarrow \text{KEYGEN}(\lambda)$ 
10:  for each  $\langle b, \text{PRF}_{\kappa_{\text{PRF}}}^C(b) \rangle$  do
11:    Compute  $k \leftarrow [\text{PRF}_{\kappa_{\text{PRF}}}^C(b) \bmod \zeta]$ 
12:    Add  $bf_k \leftarrow bf_k \cup b$ 
13:    Add  $bf_k \leftarrow bf_k \cup \text{PRP}_{\kappa_{\text{PRP}}^S}(\text{PRF}_{\kappa_{\text{PRF}}}^C(b))$ 
14:  Send  $\mathcal{T}_{\text{BF}} \leftarrow \{\text{ENC}_\kappa(bf_1), \dots, \text{ENC}_\kappa(bf_\zeta)\}$  to CLIENT

15: procedure Search( $y, \mathcal{T}_{\text{BF}}$ )
16:  if  $y \notin \mathcal{L}$  then
17:    CLIENT sets  $z \leftarrow \text{PRF}_{\kappa_{\text{PRF}}}^C(y)$ 
18:  else
19:    CLIENT sets  $z \leftarrow \{0, 1\}^\lambda$ 
20:  CLIENT computes  $k \leftarrow z \bmod \zeta$  and sends to SERVER  $z$  and
   $\text{ENC}_\kappa(bf_k)$ , where  $k \equiv z \bmod \zeta$ 
21:  SERVER decrypts  $\text{ENC}_\kappa(bf_k)$  and checks whether  $\text{PRP}_{\kappa_{\text{PRP}}^S}(z) \in$ 
   $bf_k$ . If so, SERVER outputs TRUE. Otherwise, it outputs FALSE.
22:  CLIENT adds  $\mathcal{L} \leftarrow \mathcal{L} \cup \{y\}$  if SERVER outputs FALSE

```

scheme has homomorphic properties that allows us to compute signatures of subsets of a signed set, as well as the union of two signed sets without the private key. We leverage these properties to directly compute on the signature, and implement an oblivious match. Importantly, the signature (and the ACB) is constant-sized making it storage efficient.

4.2.1. Randomizable Set-Homomorphic Signatures.

Definition 1. A randomizable set homomorphic signature over sets drawn from a universe $\mathcal{U}$ is a collection of seven polynomial-time algorithms $\Pi_{\text{RH-Sig}} = \langle \text{GEN}, \text{SIGN}, \text{VRFY}, \text{RAND}, \text{AUTH}, \text{BLIND}, \text{UNBLIND} \rangle$. The key space is $\mathcal{K}$ and the output space is $\mathcal{X}$.

- $\langle pk, sk \rangle \leftarrow \text{GEN}(1^\lambda)$: Generates a public-private key pair
- $\sigma_S \leftarrow \text{SIGN}_{sk}(S)$: On input a set $S \subseteq \mathcal{U}$, and private key sk generated by GEN, the algorithm produces a signature $\sigma \in \mathcal{X}$.
- $\{0, 1\} \leftarrow \text{VRFY}_{pk}(S, \sigma)$: On input a set $S \subseteq \mathcal{U}$, a signature σ , and the public key pk corresponding to sk , the algorithm outputs 1 if $\sigma = \text{SIGN}_{sk}(S)$. Otherwise, it outputs 0.

- $\langle v', w' \rangle \leftarrow \text{RAND}_{pk}(v)$: On input an arbitrary element from the output space $v \in \mathcal{X}$ and $r \in \mathcal{R}$, the algorithm outputs $v' \leftarrow f_{pk}(v, r)$ and $w' \leftarrow H(f_{pk}(\sigma_\emptyset, r))$
 - $\mathcal{R}$ is a space of random coins that is exponentially large in λ (defined later)
 - $f_{pk} : \mathcal{X} \times \mathcal{R} \rightarrow \mathcal{X}$ is an injective function for a fixed $r \in \mathcal{R}$, and the output of $f_{pk}(\cdot, r)$ is uniformly distributed in $\mathcal{X}$ when $r \leftarrow \mathcal{R}$.
 - $H : \mathcal{X} \rightarrow \{0, 1\}^\lambda$ is a random oracle
 - $\sigma_\emptyset = \text{SIGN}_{sk}(\emptyset)$ is the signature on the null set
- $\{v, \perp\} \leftarrow \text{AUTH}_{pk}(\mathcal{S}, \langle v', w' \rangle)$: Upon being provided $v' \in \mathcal{X}$ and $w' \in \{0, 1\}^\lambda$, the algorithm returns v if $\langle v', w' \rangle$ is an output of $\text{RAND}_{pk}(v)$ for some $v \in \mathcal{S}$. Otherwise, it returns $\perp$.
- $v' \leftarrow \text{BLIND}(v, r)$: On input $v \in \mathcal{X}$ and $r \in \mathcal{R}$, the algorithm outputs $v' \leftarrow f_{pk}(v, r)$.
- $v \leftarrow \text{UNBLIND}(v', r')$: On input $v' \in \mathcal{X}$ and $r' \in \mathcal{R}$, the algorithm outputs $v \leftarrow f_{pk}(v', r')$.

Set Homomorphism: There is an operator $\odot_{pk} : \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X}$ such that given two sets $\mathcal{S}_1$ and $\mathcal{S}_2$, signatures σ_1 and σ_2 , where $\text{VRFY}_{pk}(\mathcal{S}_1, \sigma_1) = 1$ and $\text{VRFY}_{pk}(\mathcal{S}_2, \sigma_2) = 1$, it is the case that

$$\text{VRFY}_{pk}(\mathcal{S}_1 \cup \mathcal{S}_2, \sigma_1 \odot_{pk} \sigma_2) = 1. \quad (1)$$

There is also an operator $\ominus_{pk} : \mathcal{X} \times \Phi(\mathcal{U}) \rightarrow \mathcal{X}$, where $\Phi(\mathcal{U})$ denotes the powerset of $\mathcal{U}$, such that given σ where $\text{VRFY}_{pk}(\mathcal{S}, \sigma) = 1$ and given $\mathcal{S}' \subseteq \mathcal{S}$,

$$\text{VRFY}_{pk}(\mathcal{S} \setminus \mathcal{S}', \sigma \ominus_{pk} \mathcal{S}') = 1 \quad (2)$$

The function f is homomorphic over the signature space such that for all $\mathcal{S}' \subseteq \mathcal{U}$ and $\sigma \in \mathcal{X}$

$$f_{pk}(\sigma, r) \ominus_{pk} \mathcal{S}' = f_{pk}(\sigma \ominus_{pk} \mathcal{S}', r) \quad (3)$$

Note that (3) holds even when $\mathcal{S}' \not\subseteq \mathcal{S}$.

Remark 4.1. In our discussions we will denote a signature on the set $\mathcal{S} \subseteq \mathcal{U}$ as σ_S for notational simplicity. For any $y \in \mathcal{U}$, the computation $\sigma_B \ominus_{pk} \{y\}$ returns a valid signature on the set $\mathcal{B} \setminus \{y\}$ when $y \in \mathcal{B}$ due to Eq. (2). We denote this signature with $\sigma_{\mathcal{B} \setminus \{y\}}$. However, when $y \notin \mathcal{B}$, $\sigma_B \ominus_{pk} \{y\}$ is an element in the output space $\mathcal{X}$ that is not necessarily equal to σ_B . As will be evident later, in this case, $\sigma_B \ominus_{pk} \{y\}$ is a signature on an unpredictable element in $\mathcal{U}$; with abuse of notation we also denote this element as $\sigma_{\mathcal{B} \setminus \{y\}}$.

Security: A randomizable set homomorphic signature scheme must satisfy three security properties

- **Unforgeability:** The signature scheme must satisfy unforgeability under chosen message attacks (UF-CMA), defined by the experiment $\text{EXPT}_{\Pi_{\text{Sig}}}^{\text{uf-cma}}(\mathcal{A}_{\text{ufcma}})$ in Fig. 6.
- **Indistinguishable signatures under randomization:** For all $y \notin \mathcal{B}$, we want the output of $\text{RAND}_{pk}(\sigma_B \ominus_{pk} \{y\})$ to hide y . We formalize this property as *indistinguishable signatures under randomizations (IND-SR)*, using

$\Pi_{\text{RH-Sig}}$: Randomizable Set Homomorphic Signature

Parameters: $H : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ and $H_{\mathcal{P}} : \mathcal{U} \rightarrow \mathcal{P}$ are both random oracle, and $\mathcal{P}$ is the set of odd primes less than a semi-prime modulus N .

- **GEN**(1^λ): Select two safe primes of length λ -bits $p = 2p' + 1$ and $q = 2q' + 1$ and compute $N = pq$. Publish N , and a generator g of QR_N as the public key. The order of QR_N is the private key.
- **SIGN** $_{sk}(\mathcal{S})$: Compute $U \leftarrow \prod_{y \in \mathcal{S}} H_{\mathcal{P}}(y)$, and output signature $\sigma_{\mathcal{S}} \leftarrow g^{1/U} \bmod N$.
- **VRIFY** $_{pk}(\mathcal{S}, \sigma_{\mathcal{S}})$: Compute $U \leftarrow \prod_{y \in \mathcal{S}} H_{\mathcal{P}}(y)$ and output 1 if $\sigma_{\mathcal{S}}^U \equiv g \bmod N$. Otherwise, output 0.
- **RAND** $_{pk}(\sigma_{\mathcal{S}})$: Sample $r \leftarrow \mathbb{Z}[1, N^2]$. Output $v = f_{pk}(\sigma_{\mathcal{S}}, r) = \sigma_{\mathcal{S}}^r \bmod N$ and $w = H(g^r \bmod N)$.
- **AUTH** $_{pk}(\mathcal{S}, \langle v', w' \rangle)$: For every $v \in \mathcal{S}$, compute $U \leftarrow \prod_{y \in \mathcal{S} \setminus \{v\}} y$ and output v if $H(v^U \bmod N) = w$. Otherwise, output $\perp$.
- **BLIND** $_{sk}(v, \beta)$: On input $\beta \leftarrow \mathbb{Z}[1, N^2]$, output $(v)^{\beta^{-1}} \bmod N$
- **UNBLIND** $_{pk}(v', \beta')$: On input $\beta' \leftarrow \mathbb{Z}[1, N^2]$, output $(v')^{\beta'} \bmod N$

Figure 4: Randomizable set homomorphic signature under the strong QR-RSA assumption

a CPA-style definition such that for $y_0, y_1 \notin \mathcal{B}$, the outputs of $\text{RAND}_{pk}(\sigma_{\mathcal{B}} \ominus_{pk} \{y_0\})$ and $\text{RAND}_{pk}(\sigma_{\mathcal{B}} \ominus_{pk} \{y_1\})$ are indistinguishable. This is defined by the experiment $\text{EXPT}_{\Pi_{\text{Sig}}}^{\text{ind-sr}[b]}$ in Fig. 6.

- **Confidentiality under blinding:** A blinded signature $\text{BLIND}(\sigma_{\mathcal{B}}, \beta)$ must not reveal any item in $\mathcal{B}$, i.e., for any PPT. adversary

$$\mathbb{P} \left(\begin{array}{l} b \leftarrow \mathbf{A}(v) \\ \wedge b \in \mathcal{B} \end{array} \middle| \begin{array}{l} \langle pk, sk \rangle \leftarrow \text{GEN}(1^\lambda) \\ \sigma_{\mathcal{B}} \leftarrow \text{SIGN}_{sk}(\mathcal{B}) \\ \beta \leftarrow \mathcal{R} \\ v \leftarrow \text{BLIND}_{sk}(\sigma_{\mathcal{B}}, \beta) \end{array} \right) \leq \frac{1}{|\mathcal{U}|} + \text{negl}(\lambda)$$

4.2.2. RSA-Based Construction. Fig. 4 describes a randomizable set homomorphic signature scheme that is secure under the strong QR-RSA assumption.

Correctness: The correctness of GEN, SIGN and VRIFY is obvious. We describe how the signature scheme satisfies the homomorphic properties. All calculation are performed modulo N ; we omit this factor from the description for notational simplicity.

Let $\mathcal{S}_1, \mathcal{S}_2$ be two sets and $U_1 \leftarrow \prod_{y \in \mathcal{S}_1} H_{\mathcal{P}}(y)$, and $U_2 \leftarrow \prod_{y' \in \mathcal{S}_2} H_{\mathcal{P}}(y')$. Also, let the signatures on $\mathcal{S}_1$ and $\mathcal{S}_2$ be $\sigma_{\mathcal{S}_1} \leftarrow g^{1/U_1}$ and $\sigma_{\mathcal{S}_2} \leftarrow g^{1/U_2}$ respectively.

- **Union of sets:** We define the homomorphic operator $\odot$

$$\begin{aligned} \sigma_{\mathcal{S}_1} \odot_{pk} \sigma_{\mathcal{S}_2} &= \sigma_{\mathcal{S}_1 \cup \mathcal{S}_2} = \sigma_{\mathcal{S}_1}^b \sigma_{\mathcal{S}_2}^a = g^{b/U_1} g^{a/U_2} \\ &= g^{\frac{bU_2 + aU_1}{U_1 U_2}} = g^{\frac{\text{GCD}(U_1, U_2)}{U_1 U_2}} = g^{1/\text{LCM}(U_1, U_2)} \end{aligned}$$

Here, a, b are integers such that $aU_1 + bU_2 = \text{GCD}(U_1, U_2)$, and they can be found in polynomial time

using the extended Euclid algorithm. Due to Bezout's identity a, b always exist.

- **Difference of two sets:** We define the set difference operator $\ominus$

$$\sigma_{\mathcal{S}_1} \ominus_{pk} \sigma_{\mathcal{S}_2} = \sigma_{\mathcal{S}_1 \setminus \mathcal{S}_2} = \sigma_{\mathcal{S}_1}^{U_2} = g^{U_2/U_1}$$

, where $\mathcal{S}_2 \subseteq \mathcal{S}_1$,

- **Homomorphic property over randomization:** The set difference operator $\ominus$ is homomorphic over the randomization function.

$$\begin{aligned} f_{pk}(\sigma_{\mathcal{S}_1}, r) \ominus_{pk} \sigma_{\mathcal{S}_2} &= g^{r \cdot U_2/U_1} = \\ f_{pk}(g^{U_2/U_1}, r) &= f_{pk}(\sigma_{\mathcal{S}_1} \ominus_{pk} \sigma_{\mathcal{S}_2}, r) \end{aligned}$$

We show in App. B that the security of the scheme (UF-CMA and IND-SR) reduces to the strong QR-RSA assumption [18] in the random oracle model. Confidentiality under blinding is ensured by the fact that for $\beta \leftarrow \mathbb{Z}[1, N^2]$, $f(v, \beta) = v^\beta \bmod N$ is uniformly distributed in QR_N [32].

Distributed Key Generation & Signing: To leverage the signature scheme with multiple signers, as required to publish an ACB, we need to extend it to a distributed setting such that the auditors only have a share of the private key. Otherwise, the malicious auditors can collude with server and violate the IND-SR properties necessary to support an oblivious match. Additionally, the distributed key generation must be robust against a malicious minority of signers in our threat model. Fortunately, the signature scheme can be extended to support distributed key generation using existing techniques [2]. We define the ideal functionality $\mathcal{F}_{\text{dsign}}$ for distributed key generating and signing, and describe the protocol realizing the functionality in Fig. 7 in the Appendix.

4.2.3. Blocklist construction. We show how to create an ACB using a randomizable set-homomorphic signature scheme in Algorithm 2 and depict the process in Fig. 2b. The construction is straightforward. The auditors invoke $\mathcal{F}_{\text{dsign}}$ to jointly sign the blocklisted items. The blinded signature $\sigma_{\mathcal{B}} \leftarrow \text{BLIND}_{sk^{(d)}}((\text{SIGN}_{sk^{(d)}}(\mathcal{B}), \beta)$ and the public key $pk^{(d)}$ is the ACB (lines 2 - 6). During a search with input y , the client computes $\sigma_{\mathcal{B} \setminus \{y\}}$ using Eq. (2). Then, the client computes $\text{RAND}_{pk^{(d)}}(\sigma_{\mathcal{B} \setminus \{y\}})$ and sends it to the server. The server invokes $\text{AUTH}_{pk^{(d)}}(\mathcal{B}, \text{RAND}(\sigma_{\mathcal{B} \setminus \{y\}}))$ which returns y when $y \in \mathcal{B}$ (line 9). Otherwise, the server learns nothing due to the IND-SR property of the signature scheme.

Costs: The costs of creating and searching are as follows

- CreateBlocklist has $\mathbb{O}(n)$ cost for the auditors. The client and server have no associated costs.
- Search has $\mathbb{O}(1)$ cost for the client and $\mathbb{O}(n \log n)$ for the server when the signature is initialized with the RSA-based signature scheme [6]. The communication cost is $\mathbb{O}(1)$ due to the two QR_N elements $\langle v, w \rangle$.
- The total storage cost for the client is $\mathbb{O}(1)$

Security: The security of the ACB is directly implied by the security of the signature scheme. The ACB is confidential

Algorithm 2 Π_{sig} : Signature Based Blocklist

Require: A signature scheme $\Pi_{\text{sig}} = \langle \text{SGEN}, \text{SSIGN}, \text{SVRFY} \rangle$; a collision-resistant hash function $\{0, 1\}^* \rightarrow \{0, 1\}^\lambda$; a randomized set-homomorphic signature scheme $\Pi_{\text{RH-Sig}} = \langle \text{GEN}, \text{SIGN}, \text{VRFY}, \text{RAND}, \text{AUTH}, \text{BLIND}, \text{UNBLIND} \rangle$.

- 1: **procedure** Publish($\mathcal{B}$)
- 2: Auditors jointly sample a key pair, $\langle pk^{(d)}, sk^{(d)} \rangle$, for $\Pi_{\text{RH-Sig}}$ by invoking $\mathcal{F}_{\text{dsign}}$ where the private key is secret-shared
- 3: Auditor A_k samples a key pair $\langle pk_k, sk_k \rangle$ for Π_{sig}
- 4: Auditors invoke $\mathcal{F}_{\text{dsign}}$ to sign and blind the blocklist with the blinding factor $\beta \leftarrow \mathcal{S}[1, N^2]$: $\sigma_{\mathcal{B}} \leftarrow \text{BLIND}_{sk^{(d)}}(\text{SIGN}_{sk^{(d)}}(\mathcal{B}), \beta)$. Auditors publish the ACB $\mathcal{T}_{\text{sig}} \leftarrow \langle pk^{(d)}, \sigma_{\mathcal{B}} \rangle$, and send β and $\mathcal{B}$ to SERVER.
- 5: Auditors sign the ACB under their own private keys $\Sigma_k \leftarrow \text{SSIGN}_{sk_k}(h(\mathcal{T}_{\text{sig}}))$ and publish the signatures $\{\Sigma_k\}_{k \in [1, \eta]}$
- 6: CLIENT downloads $\mathcal{T}_{\text{sig}}$ and verifies the signature under all the auditors' public keys $\{pk_k\}_{k \in [1, \eta]}$.
- 7: **procedure** Search($y, \mathcal{T}_{\text{sig}}$)
- 8: CLIENT computes

$$\sigma_{\mathcal{B} \setminus \{y\}} \leftarrow \sigma_{\mathcal{B}} \ominus_{pk^{(d)}} \{y\}$$

$$\text{RAND}_{pk^{(d)}}(\sigma_{\mathcal{B} \setminus \{y\}}) = \langle v, w \rangle$$
 CLIENT sends $\langle v, w \rangle$ to the SERVER
- 9: Upon receiving $\langle v, w \rangle$ from CLIENT, SERVER computes $v' \leftarrow \text{UNBLIND}(v, \beta)$. If $\text{AUTH}_{pk^{(d)}}(\mathcal{B}, \langle v', w \rangle) = b \in \mathcal{B}$, then SERVER outputs TRUE. Otherwise, SERVER outputs FALSE.

due to the blinding factor β , and unforgeable due to the UFCMA guarantee of the signature scheme. The IND-SR guarantee of the signature scheme ensures that for $y \notin \mathcal{B}$, the server learns no information from $\text{Search}(y, \mathcal{T}_{\text{sig}})$. A full proof is omitted from the paper.

4.3. Encrypted Cuckoo Hash Based Blocklist

The blocklist constructions we have considered so far only support the exact search (Search) mechanism. In this section, we will design an ACB that supports threshold search (TSearch) mechanism. Intuitively, the blocklisted items are hashed into a cuckoo hash table T of size $n\gamma(1 + \epsilon_{\text{CHT}})$ with hash functions h'_1, h'_2 , where ϵ_{CHT} is a parameter that controls false negative rate of the cuckoo hash table [40]. T is stored on the client and the values in each location is individually encrypted under an additively homomorphic encryption scheme. *The hash keys are not stored alongside:* the client sees $2n\gamma(1 + \epsilon_{\text{CHT}})$ encrypted values, and knows the relation between ciphertexts and their corresponding locations in the table. The empty locations in the table are padded with encryptions of dummy values.

During a search the client performs lookups in T , and then compares its inputs with the encrypted values in the corresponding locations in the hash table, by homomorphically subtracting one from the other, and blinding the result with a random multiplicative factor. If the projections match, then after decryption, the server should observe an encryption of 0.

To hide the result of the search until a threshold is met, the client secret-shares a known value in the encrypted

results. After decryption, if the server obtains enough shares, then it can reconstruct the value and confirm. *The main challenge is that the server should know which shares have been correctly decrypted when there are at least a threshold number of them, otherwise the server should not learn any information.* We address this challenge by using a *detectable* secret sharing scheme as detailed below

4.3.1. Detectable Secret Sharing. Similar to standard (ℓ_{ss}, t_{ss}) -secret-sharing, a detectable secret sharing (DSS) scheme has two algorithms $\text{dssshare} = \langle \text{DSHARE}, \text{DCOMBINE} \rangle$.

- $\{ss_1, \dots, ss_{\ell_{ss}}\} \leftarrow \text{DSHARE}(\kappa)$ takes as input a secret κ and produces a set of secret-shares $\{ss_1, \dots, ss_{\ell_{ss}}\}$
- $\kappa/\perp \leftarrow \Pi_{\text{dss}}.\text{DCOMBINE}(\{x_1, \dots, x_{\ell_{ss}}\})$ Given a set of ℓ_{ss} elements, if at least t_{ss} of them are valid secret shares, output the secret κ . Otherwise, output $\perp$.

For security, a DSS scheme ensures that when there are less than t_{ss} valid shares of κ in $\{x_1, \dots, x_{\ell_{ss}}\}$ then nothing is revealed to an adversary *including which of the shares are correct* in the set $\{x_1, \dots, x_{\ell_{ss}}\}$.

Detectable hash function: We will make use of a primitive introduced by the Apple PSI mechanism [40] called a detectable hash function to realize a DSS scheme. Formally, a (t_{dhf}, s_{dhf}) -detectable hash function $\text{DHF} : K_{dhf} \times \mathcal{D}_{dhf} \rightarrow \mathcal{R}_{dhf}$ has the following two properties.

- 1) The function is weakly t_{dhf} -wise independent, that is the distribution of $\langle \text{DHF}_{k_{dhf}}(x_1), \dots, \text{DHF}_{k_{dhf}}(x_{t_{dhf}}) \rangle$ given $k_{dhf} \leftarrow K_{dhf}$ and $x_1, \dots, x_{t_{dhf}} \leftarrow \mathcal{D}_{dhf}$ is identical to the uniform distribution over $(\mathcal{R}_{dhf})^{t_{dhf}}$.
- 2) The function is s_{dhf} -detectable. Given a vector $\vec{z} = \langle z_1, \dots, z_{\ell_{dhf}} \rangle \in (\mathcal{R}_{dhf})^{\ell_{dhf}}$, where for $\mathcal{I} \subseteq [\ell_{dhf}]$, $z_j \leftarrow \text{DHF}_{k_{dhf}}(x_j)$ when $j \in \mathcal{I}$, and $z_j \leftarrow \mathcal{R}_{dhf}$ when $j \notin \mathcal{I}$, there is an algorithm DETECT
 - $\text{DHF.DETECT}(\vec{z}) = \mathcal{I}$ with non-negligible probability, when $|\mathcal{I}| \geq \max(t_{dhf} + 1, \ell - s_{dhf})$.
 - $\text{DHF.DETECT}(\vec{z}) = \text{Fail}$ when $|\mathcal{I}| \leq t_{dhf}$.

Intuitively, given a vector $\vec{z} \in (\mathcal{R}_{dhf})^{\ell_{dhf}}$, where at least $\max(t_{dhf} + 1, \ell_{dhf} - s_{dhf})$ components are outputs of DHF, the algorithm DETECT identifies these components. However, if less than $t_{dhf} + 1$ entries are generated by DHF, then the algorithm fails and no information is revealed about $\mathcal{I}$ due to the t_{dhf} -wise independence. k_{dhf} is only needed to compute the DHF components, but not in the detection algorithm. Details about the DHF scheme are in App. A.

Protocol for detectable secret sharing: The protocol for a DSS scheme $\Pi_{\text{dss}} = \langle \text{DSHARE}, \text{DCOMBINE} \rangle$ is in Algorithm 7. Given a secret κ , the share procedure DSHARE first secret-shares it into ℓ_{ss} shares $\{ss_1, \dots, ss_{\ell_{ss}}\} \leftarrow \text{SHARE}(\kappa)$. Then, it combines the secret shares with ℓ_{ss} components of a $(t_{ss}, \ell_{ss} - t_{ss} + 1)$ DHF: $SS \leftarrow \{\langle \text{DHF}_{k_{dhf}}(r_i), ss_i \rangle_{i \in [1, \ell_{ss}]} \text{ for } r_i \leftarrow \mathcal{D}_{dhf}$.

During reconstruction using the DCOMBINE procedure, if there are at least t_{ss} DHF components, then it informs the reconstructor of the correct shares. Otherwise, there are less than t_{ss} DHF components, and so the de-

tection algorithm fails with high probability. (Note that for correctness $\max(t_{ss}, \ell_{ss} - (\ell_{ss} - t_{ss} + 1)) = t_{ss}$).

4.3.2. Blocklist construction. We construct an ACB from the encrypted cuckoo hash table described before. The full construction is in Algorithm 3 and depicted in Fig. 2c. Each location in the table is an encrypted item in the blocklist, as decided by the cuckoo hash functions (lines 3 - 5).

During a search $\text{TSearch}(Y, \mathcal{T}_{\text{CHT}}, t)$, the client uses a detectable secret sharing scheme such that the server is able to reconstruct a secret only when $|Y \cap \mathcal{B}| \geq t$. This is enabled by encrypting the detectable secret shares of the secret under independently sampled keys for a PRG (Eq. (4)–(5)), and encrypting the PRG keys using an additively homomorphic encryption such that the server obtains a key if the client’s input matches a corresponding (encrypted) item in the table (Eq. (6)–(7)). If the server obtains at least t PRG keys, then it can decrypt t DSS components, allowing it to reconstruct the secret (lines 15 - 17).

Cost & Security: The cost of creating the cuckoo hash table with n items is $\mathcal{O}(n)$. The cost of a threshold search is $\mathcal{O}(\ell_{ss})$ for the client and $\mathcal{O}(\ell_{ss}^3)$ the server due to the DHF.DETECT algorithm. Security is implied by the IND-CPA security of the additively homomorphic encryption scheme. The secret sharing scheme, and the DHF construction are both information-theoretically secure.

5. Content Moderation Schemes

5.1. Exact Matching: Realizing $\mathcal{F}_{\text{ExactMod}}$

We describe our content moderation scheme with exact-matching capabilities in Algorithm 4. The client uploads content drawn from the universe $\mathcal{U}$, and the content hash function $\text{embed} : \mathcal{U} \rightarrow \{0, 1\}^\ell$ maps content to a ℓ -bit string. Two items u, u' are similar iff. $\text{embed}(u) = \text{embed}(u')$.

Key idea: fast path combining two ACBs: We can leverage either a Bloom filter based ACB or a signature based ACB to realize $\mathcal{F}_{\text{ExactMod}}$. In the former, the matching cost is $\mathcal{O}(1)$ but the construction has false positives. In the latter, there are no false positives (or false negatives) but the matching cost for the server is linear (in the blocklist size). This motivates the questions: *can we design a scheme that has sublinear matching costs without incurring false positives?* Our solution is to combine both designs to realize $\mathcal{F}_{\text{ExactMod}}$ without false positives and sublinear matching costs (Algorithm 4).

Creating the Blocklist:

- 1) The auditors partition the blocklisted items into τ disjoint partitions $\mathcal{B}^{(1)}, \dots, \mathcal{B}^{(\tau)}$, and invoke $\Pi_{\text{sig}}.\text{Publish}(\mathcal{B}^{(m)})$ for each $m \in [1, \tau]$, to publish a signature-based ACB for each partition $\mathcal{T}_{\text{Sig}}^{(m)}$.
- 2) The auditors invoke $\Pi_{\text{bf}}.\text{Publish}(\mathcal{B})$ to publish $\mathcal{B}^{(\text{blind})}$. The client and server invoke $\Pi_{\text{bf}}.\text{Preprocess}(\mathcal{B}^{(\text{blind})})$ and the client learns the Bloom filter based ACB $\mathcal{T}_{\text{BF}}$.

Upload: When uploading some content DATA

- 1) (*fast path*): The client derives $y \leftarrow \text{embed}(\text{DATA})$ and invokes $\Pi_{\text{bf}}.\text{Search}(y, \mathcal{T}_{\text{BF}})$. If the search returns 0, then

Algorithm 3 Π_{CHT} : Encrypted Hash Table Based Blocklist

Require: A signature scheme $\Pi_{\text{Sig}} = \langle \text{SGEN}, \text{SSIGN}, \text{SVRFY} \rangle$; a collision-resistant hash function $\{0, 1\}^* \rightarrow \{0, 1\}^\lambda$; an additively homomorphic encryption scheme $\Pi_{\text{ahe}} = \langle \text{AH_KEYGEN}, \text{AH_ENC}, \text{AH_DEC} \rangle$; a $(2\ell, t)$ detectable secret sharing scheme $\text{dsshare} = \langle \text{DSHARE}, \text{DCOMBINE} \rangle$ such that DSHARE outputs elements in $\mathcal{R}_{dss}$; a pseudorandom generator $\text{PRG} : \mathcal{K}_{\text{PRG}} \rightarrow \mathcal{R}_{dss}$

- 1: **procedure** Publish($\mathcal{B}$)
- 2: Auditor A_k samples a key pair $\langle pk_k, sk_k \rangle$ for Π_{Sig}
- 3: The auditors initialize a cuckoo hash table T with $(1 + \epsilon_{\text{CHT}}) \cdot n$ slots, and samples two hash function $h'_1, h'_2 : \{0, 1\}^\lambda \rightarrow [(1 + \epsilon_{\text{CHT}}) \cdot n]$.
- 4: The auditors samples a key pair pk_e, sk_e for Π_{ahe}
- 5: For $b \in \mathcal{B}$, the auditors place $\text{AH_ENC}_{pk_e}(b)$ at slot $T[h'_1(b)]$ or $T[h'_2(b)]$ using cuckoo hashing. At each empty slot in the table, store $\text{AH_ENC}_{pk_e}(\mathbf{0})$.
- 6: The auditors sign the ACB $\mathcal{T}_{\text{CHT}} \leftarrow T$ under their own private keys and publish $\{\text{SSIGN}_{sk_k}(h(\mathcal{T}_{\text{CHT}}))\}_{k \in [1, \eta]}$. The auditors provide $\mathcal{B}$ and sk_e to SERVER.
- 7: **procedure** TSearch($Y \leftarrow \{y_i\}_{i \in [1, \ell]}, \mathcal{T}_{\text{CHT}}, t$)
- 8: $\triangleright$ Client secret-shares a secret value and encrypts the shares $\triangleleft$
- 9: CLIENT invokes $SS = \{s_1^*, \dots, s_{2\ell}^*\} \leftarrow \text{DSHARE}(\mathbf{0})$
- 10: CLIENT samples 2ℓ PRG keys $\kappa_1, \dots, \kappa_{2\ell} \leftarrow \mathcal{K}_{\text{PRG}}$
- 11: **for** $y_j \in Y, \dots, y_\ell$ **do**
- 12: CLIENT retrieves c_1 from $T[h'_1(y_j)]$ and c_2 from $T[h'_2(y_j)]$ and computes

$$u_j \leftarrow r_{1,j} \odot_{pk_e} (c_1 +_{pk_e} (-y_j)) +_{pk_e} \text{Enc}_{pk_e}(\kappa_j) \quad (4)$$

$$u_{\ell+j} \leftarrow r_{2,j} \odot_{pk_e} (c_2 +_{pk_e} (-y_j)) +_{pk_e} \text{Enc}_{pk_e}(\kappa_{\ell+j}) \quad (5)$$

$$u_j^* \leftarrow s_j^* \oplus \text{PRG}(\kappa_j) \quad (6)$$

$$u_{\ell+j}^* \leftarrow s_{\ell+j}^* \oplus \text{PRG}(\kappa_{\ell+j}) \quad (7)$$

where $r_{1,j}, r_{2,j}$ are random scalars. The encrypted calculations in Eq. (4)–(5) are performed using the additively homomorphic operators defined in Sec. 3.
- 13: CLIENT randomly shuffles the set $\{\langle u_j, u_j^* \rangle\}_{j \in [1, 2\ell]}$ and sends to SERVER
- 14: $\triangleright$ Upon receiving $\{\langle u_j, u_j^* \rangle\}_{j \in [1, 2\ell]}$ from CLIENT $\triangleleft$
- 15: SERVER decrypts u_j for all $j \in [1, 2\ell]$ using sk_e , and obtains $\{\kappa'_1, \dots, \kappa'_{2\ell}\}$
- 16: SERVER computes $SS' \leftarrow \{s' : u_j^* \oplus \kappa'_j, j \in [1, 2\ell]\}$
- 17: If $\text{DCOMBINE}(SS') = \perp$, then SERVER outputs FALSE. Otherwise it outputs TRUE.

the client and server can abort because this means $y \notin \mathcal{B}$. The server will store the encrypted content.

- 2) (*fine-grained check*): Otherwise, let bf_k be the Bloom filter such that $\text{PRP}_{\kappa_{\text{PRG}}}(\text{PRF}_{\kappa_{\text{PRF}}}(y)) \in bf_k$; recall that the server learns bf_k in $\Pi_{\text{bf}}.\text{Search}$. The server identifies the partitions containing the items included in bf_k .

$$\mathcal{I} \leftarrow \{m : b \in bf_k \wedge b \in \mathcal{B}^{(m)}\}$$

, and sends the set of indices $\mathcal{I}$ to the client

- 3) Upon receiving $\mathcal{I}$, the client invokes $\Pi_{\text{sig}}.\text{Search}(y, \mathcal{T}_{\text{Sig}}^{(m)})$ for all $m \in \mathcal{I}$. If any invocation returns y to the server, then $y \in \mathcal{B}$; the server rejects the encrypted content and aborts. Otherwise, $y \notin \mathcal{B}$ and the server accepts the content and informs the client.

Algorithm 4 Π_{EM} : content moderation with exact matching

```

1: ▷ Partition the blocklist and publish signature-based ACB          ◁
2: Auditors distribute  $\mathcal{B}$  into  $\tau$  disjoint buckets  $\mathcal{B}^{(1)}, \dots, \mathcal{B}^{(\tau)}$ 
3: for each  $m \in [1, \tau]$  do
4:   Auditors invoke  $\Pi_{\text{sig}}$ .Publish( $\mathcal{B}^{(m)}$ ) and publish the signature-
     based ACB  $\mathcal{T}_{\text{Sig}}^{(m)} = \langle pk^{(d)}, \sigma_{\mathcal{B}^{(m)}} \rangle$ 
5:   Auditors sign and publish  $\{\text{SSIGN}_{sk_k}(h(\mathcal{T}_{\text{Sig}}^{(m)}))\}_{k \in [1, \eta]}$ 

6: ▷ Publish blinded blocklist for Bloom filter based ACB          ◁
7: Auditors invoke  $\Pi_{\text{bf}}$ .Publish( $\mathcal{B}$ ) and publish a blinded blocklist
    $\mathcal{B}^{(\text{blind})}$  and the signatures  $\{\text{SSIGN}_{sk_k}(h(\mathcal{B}^{(\text{blind})}))\}_{k \in [1, \eta]}$ 

8: ▷ Client and server preprocess blinded blocklist              ◁
9: CLIENT downloads  $\mathcal{B}^{(\text{blind})}$ ,  $\{\text{SSIGN}_{sk_k}(h(\mathcal{B}^{(\text{blind})}))\}_{k \in [1, \eta]}$ ,
    $\{\mathcal{T}_{\text{Sig}}^{(m)}\}_{m \in [1, \tau]}$ ,  $\{\{\text{SSIGN}_{sk_k}(h(\mathcal{T}_{\text{Sig}}^{(m)}))\}_{k \in [1, \eta]}\}_{m \in [1, \tau]}$ , and
   verifies the signatures. If the signatures do not verify, then CLIENT
   aborts.
10: CLIENT and SERVER invoke  $\Pi_{\text{bf}}$ .Preprocess( $\mathcal{B}^{(\text{blind})}$ ). CLIENT stores
     $\mathcal{T}_{\text{BF}} \leftarrow \{\text{ENC}_K(bf_1), \dots, \text{ENC}_K(bf_\zeta)\}$ 

11: ▷ Matching against blocklist during uploads                    ◁
12: CLIENT sends  $\langle \text{ID}, \text{ENC}_K_f(\text{DATA}) \rangle$  to SERVER
13: CLIENT computes  $y \leftarrow \text{embed}(\text{DATA})$  and  $\Pi_{\text{bf}}$ .Search( $y, \mathcal{T}_{\text{BF}}$ ).
14: if  $\Pi_{\text{bf}}$ .Search( $y, \mathcal{T}_{\text{BF}}$ ) = 0 then                                ▷ Fast path
15:   SERVER outputs  $\langle \text{ID}, \text{detected} = \text{FALSE}, \emptyset \rangle$  and aborts.
16: else                                                                ▷ Fine-grained check
17:   SERVER sends  $\mathcal{I} \leftarrow \{m : b \in bf_k \wedge b \in \mathcal{B}^{(m)}\}$  to CLIENT.
18:   for each  $m \in \mathcal{I}$  do
19:     CLIENT invokes  $\Pi_{\text{sig}}$ .Search( $y, \mathcal{T}_{\text{Sig}}^{(m)}$ )
20:     if  $\Pi_{\text{sig}}$ .Search( $y, \mathcal{T}_{\text{Sig}}^{(m)}$ ) =  $y$  then
21:       SERVER outputs  $\langle \text{ID}, \text{detected} = \text{TRUE}, y \rangle$  and aborts
22:     else
23:       SERVER outputs  $\langle \text{ID}, \text{detected} = \text{FALSE}, \emptyset \rangle$ .

```

Costs: The preprocessing for $\mathcal{T}_{\text{BF}}$ requires a one-time $\mathcal{O}(n)$ compute and communication cost for the client and the server. The client also requires $\mathcal{O}(n + \tau)$ storage.

During an upload, the client and server have $\mathcal{O}(1)$ compute cost (with high probability) when the uploaded content does not match an item in the blocklist, i.e., fast path verification. Alternatively, the client has $\mathcal{O}(|\mathcal{I}|) \in \mathcal{O}(\log n)$ compute cost, and the server has $\mathcal{O}(n)$ cost to identify $\mathcal{I}$, when the client's uploaded file matches an item on the blocklist. The communication cost for an upload is $\mathcal{O}((n/\tau) + \log n)$ for sending an encrypted Bloom filter, and $\mathcal{O}(\log n)$ randomized signatures for a fine-grained check. If $\tau \in \mathcal{O}(n)$, then the cost of an upload is $\mathcal{O}(\log n)$.

Theorem 5.1. Assuming there are protocols securely realizing $\mathcal{F}_{\text{dsign}}$ and $\mathcal{F}_{\text{authOPRF}}$, Π_{EM} securely realizes $\mathcal{F}_{\text{ExactMod}}$

The proof is available in the Appendix.

5.2. Fuzzy Matching: Realizing $\mathcal{F}_{\text{FuzzyMod}}$

We will describe a content moderation scheme realizing $\mathcal{F}_{\text{FuzzyMod}}$ where the metric is the Hamming distance over bit vectors of length ℓ ; several image embeddings [29, 49, 55] and existing work on fuzzy content moderation [28] use Hamming distance as the metric. Our protocol will detect matches of the form $\langle \vec{b}, \vec{y} \rangle$ such that $\text{dist}(\vec{b}, \vec{y}) \leq t$ where $\text{dist}(\cdot, \cdot)$ is the Hamming distance.

Key Idea: Using projections for a threshold match: Comparing an input against the blocklist is challenging when we need to implement a fuzzy match. The problem is that to accurately detect a match, we must compute $\text{dist}(\vec{b}, \vec{y})$ for all $\vec{b} \in \mathcal{B}$. One solution is to use a generic fuzzy membership test (PMT) or a fuzzy private set intersection (PSI) protocol [22, 41, 52], but they have high computational costs, and cannot scale to the latency requirements of a file server.

We take a different approach by leveraging the fact that our functionality has strictly weaker requirements: i) it tolerates false positives and false negatives, and ii) it does not output the matching items to the server/client. Thus, we can design a protocol with lower costs. The key idea is to use projections of the blocklist vectors as the matching criteria: two vectors are considered close if they have identical projections under a threshold number of projection functions. Obviously, leaking the projections will allow the server to learn information about the encrypted content. Therefore, we only reveal the *number of matching projections* when they are above the threshold, but not the projections themselves. We describe the projection mechanism below.

Definition 2. For a metric space $\mathcal{M}$, a locality-preserving fingerprint of an element m in the set $\mathcal{B} \subseteq \mathcal{M}$ comprises its projections under uniformly sampled functions $\pi_1, \dots, \pi_\gamma : \mathcal{M} \rightarrow \mathcal{S}$, $FP(m) \leftarrow \{h(\pi_1(m), \pi_1) \dots, h(\pi_\gamma(m), \pi_\gamma)\}$, where $h : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a collision-resistant hash function, such that given any $m' \in \mathcal{M}$,

$$\mathbb{P}(\exists j \in [1, \gamma], \pi_j(m) = \pi_j(m') \mid \text{dist}(m, m') \leq t) \leq \epsilon_{\text{fpr}}^\gamma \quad (8)$$

$$\mathbb{P}(\exists j \in [1, \gamma], \pi_j(m) \neq \pi_j(m') \mid \text{dist}(m, m') > t) \leq \epsilon_{\text{fpr}}^\gamma \quad (9)$$

where, ϵ_{fpr} and ϵ_{fpr} are the false positive rate and false negative rate under each projection function.

Intuitively, Eq. (8) ensures that two items close in the metric space agree on at least one of the projections with high probability. Eq. (9) implies that two items far apart in the metric space do not agree on *all* their projections with high probability. Since the projection functions are selected randomly, the overall false positives and false negatives over the projections are binomially distributed. Specifically,

Corollary 5.1. Let $t \in \mathbb{R}^{++}$ be a distance threshold, and $\alpha : \mathbb{R}^{++} \rightarrow [1, \gamma]$ is a scaling function. Also, let $t_D \leftarrow \alpha(t)$. Given two elements $m, m' \in \mathcal{M}$ and a detection threshold,

$$\begin{aligned}
\mathbb{P}(\text{BAD}_{FN}) &= \mathbb{P}\left(\left| \frac{|FP(m) \cap FP(m')|}{\gamma - t_D + 1} \right| \mid \text{dist}(m, m') \leq t\right) \\
&\leq 1 - \sum_{i=0}^{\gamma - t_D} \binom{\gamma}{i} \epsilon_{\text{fpr}}^i (1 - \epsilon_{\text{fpr}})^{\gamma - i} \\
\mathbb{P}(\text{BAD}_{FP}) &= \mathbb{P}\left(\left| \frac{|FP(m) \cap FP(m')|}{\geq t_D} \right| \mid \text{dist}(m, m') > t\right) \\
&\leq \sum_{i=t_D}^{\gamma} \binom{\gamma}{i} \epsilon_{\text{fpr}}^i (1 - \epsilon_{\text{fpr}})^{\gamma - i}
\end{aligned}$$

Algorithm 5 Π_{FM} : content moderation with exact matching

```

1: ▷ Create a CHT based blocklist using locality-preserving fingerprints ◁
2: for  $\vec{b} \in \mathcal{B}$  do
3:   Auditors compute the the locality-preserving fingerprints
    $\{FP(\vec{b})\}_{\vec{b} \in \mathcal{B}}$  using  $\gamma$  randomly sampled functions  $\pi_1, \dots, \pi_\gamma$ 
4: Auditors invoke  $\Pi_{\text{cht}}.\text{Publish}(\{FP(\vec{b})\}_{\vec{b} \in \mathcal{B}})$  and publish  $\mathcal{T}_{\text{CHT}}$  and
    $\{\text{SSIGN}_{sk_k}(h(\mathcal{T}_{\mathcal{B}}))\}_{k \in [1, \eta]}$ .

5: ▷ Threshold matching against blocklist during uploads ◁
6: CLIENT sends  $\langle \text{ID}, \text{ENC}_{\kappa_f}(\text{DATA}) \rangle$  to SERVER
7: CLIENT computes  $\vec{y} \leftarrow \text{embed}(\text{DATA})$  and  $FP(\vec{y})$  under the projec-
   tion functions  $\pi_1, \dots, \pi_\gamma$ .
8: CLIENT and SERVER invoke  $\Pi_{\text{cht}}.\text{TSearch}(FP(\vec{y}), \mathcal{T}_{\text{CHT}}, t_D)$ .
9: if  $\Pi_{\text{cht}}.\text{TSearch}(FP(\vec{y}), \mathcal{T}_{\text{CHT}}) = 1$  then
10:   SERVER adds ID to flagged content list  $\mathcal{D}$ 
11:   SERVER outputs  $\langle \text{ID}, \text{detected} = \text{TRUE}, \phi \rangle$ .
12: else
13:   SERVER outputs  $\langle \text{ID}, \text{detected} = \text{FALSE} \rangle$ 

```

The proof is available in the full version [1].

Construction for the Hamming distance: Chongchitmate et al. [14] describe a construction for a locality-preserving fingerprint using uniformly sampled functions $\pi_1, \dots, \pi_\gamma$ where $\pi_j : \{0, 1\}^\ell \rightarrow \{0, 1\}^\lambda$. Each function specifies a random subset of bits from the input vector, and injectively maps it to a λ -bit string. Consider a set $\mathcal{B} \subseteq \{0, 1\}^\ell$.

- 1) Create γ key-value stores $\mathcal{S}_1, \dots, \mathcal{S}_\gamma$ where the keys are the vectors in $\mathcal{B}$.
- 2) For each function π_j and each element $\vec{b} \in \mathcal{B}$
 - a) Evaluate $h(\pi_j(\vec{b}), j)$ where $h : \{0, 1\}^{\lambda + \log \gamma} \rightarrow \{0, 1\}^\lambda$ is a collision-resistant hash function.
 - b) Insert $h(\pi_j(\vec{b}), j)$ in the key-value store $\mathcal{S}_j$ at index $\mathcal{S}_j[\vec{b}]$ unless $\pi_j(\vec{b})$ is already there in $\mathcal{S}_j$. If so, assign a random value from the range $\{0, 1\}^\lambda$.

The set $FP_{\vec{b}} \leftarrow \{\mathcal{S}_1[\vec{b}], \dots, \mathcal{S}_\gamma[\vec{b}]\}$ is a locality-preserving fingerprint of $\vec{b}$. We refer to [14] for further details about how to select γ for specific values of BAD_{FN} and BAD_{FP} . If all the vector embeddings in $\mathcal{B}$ are at least $> 2t$ distance apart, then γ is sublinear in n ; most embedding functions satisfy this property [28, 31].

5.2.1. Protocol design. A locality-preserving fingerprint provides a construction realizing $\mathcal{F}_{\text{FuzzyMod}}$ (Algorithm 5).

- 1) The auditors publish an encrypted hash table based ACB by invoking $\Pi_{\text{cht}}.\text{Publish}(\{FP(\vec{b})\}_{\vec{b} \in \mathcal{B}})$ with the locality-preserving fingerprints of the blocklist vectors.
- 2) During upload, the client computes $\vec{y} \leftarrow \text{embed}(\text{DATA})$ and invokes $\Pi_{\text{cht}}.\text{TSearch}(FP(\vec{y}), \mathcal{T}_{\text{CHT}}, t_D)$. The search returns TRUE when $|FP(\vec{y}) \cap \mathcal{T}_{\text{CHT}}| \geq t_D$. Due to Corollary 5.1, there is some $\vec{b} \in \mathcal{B}$ such that $\text{dist}(\vec{y}, \vec{b}) \leq t$ with probability at least $1 - \mathbb{P}(\text{BAD}_{FP})$.
- 3) When $\Pi_{\text{cht}}.\text{TSearch}(FP(\vec{y}), \mathcal{T}_{\text{CHT}}, t_D) = \text{TRUE}$, the server adds the ID to a *flagged content list* $\mathcal{D}$.

Once the flagged content list for an account exceeds a certain threshold t_R , the server may suspend the account and/or delete the flagged files. Given BAD_{FP} , the server can estimate the expected number of false positives in a certain

number of uploads, and check if the client's uploaded content exceeds this number by a large margin. This approach was also followed by the Apple PSI system, although it additionally revealed the content key to the server, which we avoid for a more conservative definition of privacy.

Cost & Security : Creating the ACB has $\mathcal{O}(n)$ cost, and as discussed in Sec. 4, the cost of a threshold search is $\mathcal{O}(t_D)$ for the client and $\mathcal{O}(\gamma^3)$ for the server.

Theorem 5.2. *Assuming there is an IND-CPA secure additively homomorphic encryption scheme, Π_{FM} securely realizes $\mathcal{F}_{\text{ExactMod}}$ with false positive rate BAD_{FP} and false negative rate BAD_{FN} .*

The proof is available in the Appendix.

6. Evaluation

All our protocols have been implemented and available anonymously [1]. We implemented the protocols in C++ and Python. Group arithmetic was implemented using the NTL library⁷. For the OPRF and the partially homomorphic encryption scheme, we used the *oblivious* library⁸, and the *lightPHE* library [46] respectively. We used the exponential Elgamal encryption scheme for the additively-homomorphic encryption over the ED25519 elliptic curve. All parameters were configured to provide 112 bits of security. Benchmarks were performed with a single server running on an Amazon t2.xlarge instance with 8vCPUs and 32GB of memory. Our implementations are single-threaded. Clients were run on a t2.large instance with 2vCPUs and 8GB of memory.

6.1. Exact Matching

We conducted experiments for Π_{EM} using blocklists containing random SHA256 hashes. All tests were run 10 times and results were averaged across all runs. We compared our protocol with the PEMC protocol [28]⁹. *We stress that our protocol works against a malicious server in contrast to the semi-honest server in case of PEMC.*

Publishing the blocklist: Fig. 5a shows the compute costs for publishing the blocklist. Each blocklist partition has at most 64 items. Creating a blocklist with a million items takes roughly 170 seconds with up to 10 auditors. The total storage cost for the client is $\mathcal{O}(|\mathcal{T}_{\text{BF}}| + |\mathcal{T}_{\text{Sig}}|)$; for a blocklist with a million items, this is roughly 8MB.

Preprocessing: Fig. 5b shows the compute time required for preprocessing the blocklist. In our implementation, the PRF of choice is the 2HashDH over curve25519¹⁰. The number of bins in our implementation was fixed to $n/\log n$. For a million blocklisted items, preprocessing takes roughly 21 seconds for the client and 36 seconds for the server.

7. <https://libntl.org>

8. <https://pypi.org/project/oblivious/>

9. Their implementation is not publicly available but the paper reports benchmarks on a 6-core system with 32GB memory. For a fair comparison, we benchmarked our protocols on a similarly-provisioned system.

10. <https://www.ietf.org/archive/id/draft-irtf-cfrg-ristretto255-00.html>

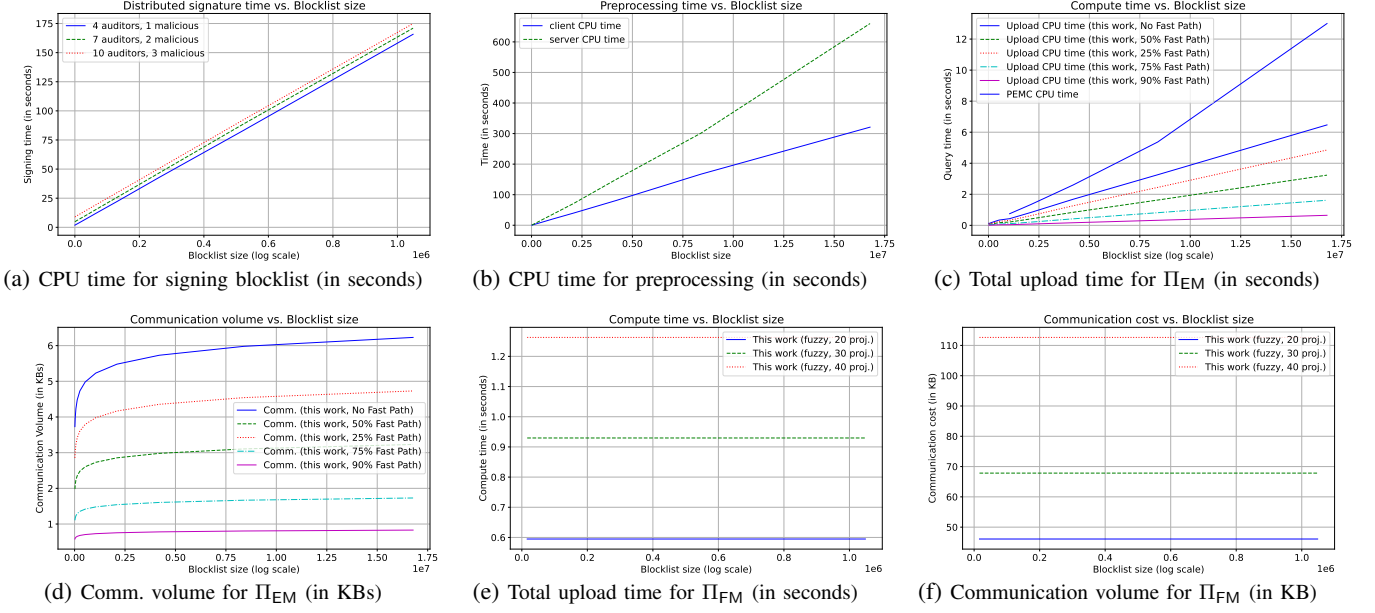


Figure 5: Costs for content moderation with exact and fuzzy matching

DB size	Our protocol Π_{FM}		PAMC [28, Table 8]	
	Time (in sec.)	Comm. (in KB)	Time (in sec.)	Comm. (in KB)
1048576	0.59	20	27.5	508
2097152	0.59	20	27.5	586
4194304	0.59	20	27.7	742
8388608	0.59	20	28.3	1053.9

Table 2: Comparison of Π_{FM} against PAMC [28].

Uploads: We evaluated the upload time in Π_{EM} by varying the fraction of queries that require a fast path verification (Fig. 5c). The reported times are averaged over 1000 queries. In the most conservative case, where every upload results in a fine-grained check, the protocol requires roughly 430 milliseconds for a blocklist with a million items. In contrast, the PEMC protocol requires 750 milliseconds. Moreover, when the fraction of queries served through a fast path verification increases, the compute times reduce proportionally. This feature is absent from the PEMC protocol, which runs a full check for every uploaded file. Assuming that a large fraction (90%) of the content will not be blocked, Π_{EM} achieves verification times of 43 ms per upload on average, a **12 $\times$ compute cost improvement over PEMC**.

Fig. 5d shows the communication costs of Π_{EM} . The communication required for a 1 million-item blocklist is roughly 6KB per upload. In comparison, PEMC requires 394KB since it performs a PIR over the blocklist. Thus, Π_{EM} has **50 $\times$ lower communication volume than PEMC**.

6.2. Fuzzy Matching

We conducted experiments for our fuzzy matching protocol Π_{FM} using a blocklist with random 256-bit vectors that simulate perceptual hashes. We varied the number of projections between 20 and 40. 20 projections allow

a false positive rate $BAD_{FP} \leq 0.02$ and false negative rate $BAD_{FN} \leq 0.07$; increasing the number of projections improves on BAD_{FN} , while keeping the BAD_{FP} roughly the same. With 20 projections, the storage cost for a million-item blocklist is 2.5GB, less than 5% of the capacity of a typical commercially available smartphone.

Uploads: Figs. 5e–5f shows how compute time and communication volume scale with blocklist size (n) and number of projections (γ) during uploads. As expected, n does not impact either metric; the costs only depend on γ .

We also compared against the private approximate matching (PAMC) protocol [28] that only tolerates a semi-honest server and has a higher false negative rate ≥ 0.17 . The experiments were performed with 20 projections for Π_{FM} . Table 2 presents the results. We observe that our protocol is faster (**by a factor of 46 $\times$**) and requires less communication (**by at least a factor of 25 $\times$**). The protocol in [28] does not have any false positives, but this comes at the cost of prohibitively high latency and a more restrictive threat model. We tolerate a small number of false positives (without leaking any other information) for better performance while also tolerating a malicious server.

7. Conclusion

This paper described protocols for detecting harmful content in encrypted files stored on an untrusted server, by comparing against a verified, common blocklist published by a distributed set of auditors. When computing exact hash matches against the blocklist, our protocol outperforms the state of the art both in terms of server and client costs. The protocol for fuzzy matching is secure against a malicious server, and features faster compute times than the state of the art that operate under a model with a semi-honest server.

References

- [1] “Code repository for auditable content moderation.” [Online]. Available: <https://anonymous.4open.science/r/content-mod-code-2125/README.md>
- [2] J. Algesheimer, J. Camenisch, and V. Shoup, “Efficient computation modulo a shared secret with application to the generation of shared safe-prime products,” in *CRYPTO*. Springer, 2002, pp. 417–432.
- [3] I. Amazon, “An update on amazon’s efforts to combat child sexual abuse material,” <https://www.aboutamazon.com/news/policy-news-views/amazon-csam-transparency-report-2023>, 2023.
- [4] Apple, Inc., “CSAM detection – technical summary,” https://www.apple.com/child-safety/pdf/CSAM_Detection_Technical_Summary.pdf, 2021.
- [5] G. Asharov and Y. Lindell, “A full proof of the bgw protocol for perfectly secure multiparty computation,” *Journal of Cryptology*, 2017.
- [6] G. Ateniese, E. De Cristofaro, and G. Tsudik, “(if) size matters: size-hiding private set intersection,” in *PKC*. Springer, 2011, pp. 156–173.
- [7] K. Bartley, “Big data statistics: How much data is there in the world?” <https://riverty.io/blog/big-data-statistics-how-much-data-is-there-in-the-world/>, 2025.
- [8] BBC, “Apple criticised for system that detects child abuse,” <https://www.bbc.com/news/technology-58124495>, 2022.
- [9] A. Bhowmick, D. Boneh, S. Myers, K. Talwar, and K. Tarbe, “The Apple PSI system,” https://www.apple.com/child-safety/pdf/Apple_PSI_System_Security_Protocol_and_Analysis.pdf, 2021.
- [10] R. Canetti and G. Kaptchuk, “The broken promise of Apple’s announced forbidden-photo reporting system – and how to fix it,” <https://www.bu.edu/riscs/2021/08/10/apple-csam/>, 2026.
- [11] —, “The broken promise of apple’s announced forbidden-photo reporting system – and how to fix it,” Center for Reliable Information Systems & Cyber Security, Boston University, August 2021. [Online]. Available: <https://www.bu.edu/riscs/2021/08/10/apple-csam/>
- [12] D. Catalano, R. Gennaro, and S. Halevi, “Computing inverses over a shared secret modulus,” in *EUROCRYPT*. Springer, 2000.
- [13] K. Choi, A. Manoj, and J. Bonneau, “Sok: Distributed randomness beacons,” in *IEEE S&P*, 2023.
- [14] W. Chongchitmate, S. Lu, and R. Ostrovsky, “Approximate PSI with near-linear communication,” *Cryptology ePrint Archive*, 2024.
- [15] CNN, “Apple abandons controversial plan to check ios devices and icloud photos for child abuse imagery,” <https://www.cnn.com/2022/12/08/tech/apple-csam-tool>, 2022.
- [16] S. Colombo, K. Nikitin, H. Corrigan-Gibbs, D. J. Wu, and B. Ford, “Authenticated private information retrieval,” in *USENIX Security*, 2023.
- [17] D. Coppersmith and M. Sudan, “Reconstructing curves in three (and higher) dimensional space from noisy data,” in *ACM STOC*, 2003, pp. 136–142.
- [18] R. Cramer and V. Shoup, “Signature schemes based on the strong RSA assumption,” *ACM Transactions on Information and System Security*, 2000.
- [19] L. Du, A. T. Ho, and R. Cong, “Perceptual hashing for image authentication: A survey,” *Signal Processing: Image Communication*, 2020.
- [20] D. Fiore, R. Gennaro, and V. Pastro, “Efficiently verifiable computation on encrypted data,” in *ACM CCS*, 2014.
- [21] N. C. for Missing and E. Children, “Missing children posters,” <https://www.missingkids.org/home>, 2026.
- [22] G. Garimella, M. Rosulek, and J. Singh, “Structure-aware private set intersection, with applications to fuzzy matching,” in *CRYPTO*. Springer, 2022.
- [23] Google, Inc., “Fighting child sexual abuse online,” <https://protectingchildren.google/introduction>, 2026.
- [24] S. Jarecki, A. Kiayias, and H. Krawczyk, “Round-optimal password-protected secret sharing and t-pake in the password-only model,” in *ASIACRYPT*. Springer, 2014, pp. 233–253.
- [25] P. Jauernig, A.-R. Sadeghi, and E. Stapf, “Trusted execution environments: properties, applications, and challenges,” *IEEE Security & Privacy*, 2020.
- [26] R. Johnson, D. Molnar, D. Song, and D. Wagner, “Homomorphic signature schemes,” in *Topics in Cryptology – CT-RSA 2002*, ser. Lecture Notes in Computer Science, vol. 2271. Springer, 2002, pp. 244–262.
- [27] J. Katz and Y. Lindell, *Introduction to modern cryptography: principles and protocols*. Chapman and hall/CRC, 2007.
- [28] A. Kulshrestha and J. Mayer, “Identifying harmful media in end-to-end encrypted communication: Efficient private membership computation,” in *USENIX Security*, 2021.
- [29] D. Leblanc-Albarel and B. Preneel, “Black-box collision attacks on widely deployed perceptual hash functions,” *Cryptology ePrint Archive*, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1869>
- [30] J. Lu, V. E. Liang, and J. Zhou, “Deep hashing for scalable image search,” *IEEE Transactions on Image Processing*, vol. 26, no. 5, pp. 2352–2367, 2017.
- [31] S. McKeown and W. J. Buchanan, “Hamming distributions of popular perceptual hashing techniques,” *Forensic Science International: Digital Investigation*, vol. 44, p. 301509, 2023.
- [32] D. Micciancio, “The RSA group is pseudo-free,” in *EUROCRYPT 2005*, ser. Lecture Notes in Computer Science, vol. 3494. Springer, 2005, pp. 387–403.
- [33] S. Morgan, “The world will store 200 zettabytes of data by 2025,” <https://cybersecurityventures.com/the-world-will-store-200-zettabytes-of-data-by-2025/>, 2024.
- [34] J. Ning, G. S. Poh, J. Loh, J. Chia, and E. Chang, “PrivDPI: Privacy-preserving encrypted traffic inspection with reusable obfuscated rules,” in *ACM CCS*, 2019.

- [35] J. Ning, X. Huang, G. S. Poh, S. Xu, J. Loh, J. Weng, and R. H. Deng, “Pine: Enabling privacy-preserving deep packet inspection on TLS with rule-hiding and fast connection establishment,” in *European Symposium on Research in Computer Security*, 2020.
- [36] U. D. of Justice, “International statement: End-to-end encryption and public safety,” <https://www.justice.gov/archives/opa/pr/international-statement-end-end-encryption-and-public-safety>, 2020.
- [37] —, “Man stored child pornography on google account, sentenced to 14 years in federal prison,” <https://www.justice.gov/usao-wdtx/pr/man-stored-child-pornography-google-account-sentenced-14-years-federal-prison>, 2023.
- [38] —, “Independence man sentenced for child pornography,” <https://www.justice.gov/usao-wdmo/pr/independence-man-sentenced-child-pornography-1>, 2023.
- [39] E. Ong and J. Kubiawicz, “Optimizing robustness while generating shared secret safe primes,” in *PKC*. Springer, 2005.
- [40] B. Pinkas, “The private set intersection (PSI) protocol of the apple CSAM detection system,” <https://decentralizedthoughts.github.io/2021-08-29-the-private-set-intersection-psi-protocol-of-the-apple-csam-detection-system/>, 2021.
- [41] L. Piske, J. Singh, N. Trieu, V. Kolesnikov, and V. Zikas, “Distance-aware ot with application to fuzzy psi,” in *ACM CCS*, 2025.
- [42] M. Raginsky and S. Lazebnik, “Locality-sensitive binary codes from shift-invariant kernels,” in *Advances in Neural Information Processing Systems*, 2009.
- [43] E. Rescorla, “Overview of apple’s client-side CSAM scanning,” <https://educatedguesswork.org/posts/apple-csam-intro/>, August 2021.
- [44] S. Scheffler and J. Mayer, “SoK: Content moderation for end-to-end encryption,” in *PETS*, 2023.
- [45] S. Scheffler, A. Kulshrestha, and J. Mayer, “Public verification for private hash matching,” in *IEEE S&P*. IEEE, 2023.
- [46] S. I. Serengil and A. Ozpinar, “Lightphe: Integrating partially homomorphic encryption into python with extensive cloud environment evaluations,” *arXiv preprint arXiv:2408.05219*, 2025, doi: 10.48550/arXiv.2408.05219. [Online]. Available: <https://arxiv.org/abs/2408.05219>.
- [47] S. Shan, J. Cryan, E. Wenger, H. Zheng, R. Hanocka, and B. Y. Zhao, “Glaze: Protecting artists from style mimicry by {Text-to-Image} models,” in *USENIX Security*, 2023.
- [48] J. Sherry, C. Lan, R. Ada Popa, and S. Ratnasamy, “BlindBox: Deep packet inspection over encrypted traffic,” in *ACM SIGCOMM*, 2015, pp. 213–226.
- [49] A. Singh, M. Z. Khan, S. Sharma, and N. C. Debnath, “Perceptual hashing algorithms for image recognition,” in *International Conference on Advanced Intelligent Systems and Informatics*. Springer, 2025.
- [50] L. Struppek, D. Hintersdorf, D. Neider, and K. Kersting, “Learning to break deep perceptual hashing: The

use case neuralhash,” in *ACM Conference on Fairness, Accountability, and Transparency*, 2022.

- [51] United States Department of Justice, “International statement: End-to-end encryption and public safety,” <https://www.justice.gov/archives/opa/pr/international-statement-end-end-encryption-and-public-safety>, 2020.
- [52] A. van Baarsen and S. Pu, “Fuzzy private set intersection with large hyperballs,” in *EUROCRYPT*, 2024.
- [53] J. Wen, J. Liu, A. Wu, and J. Weng, “A general framework for matching pattern hiding in deep packet inspection,” in *22nd International Conference on Information Security Applications*, ser. Lecture Notes in Computer Science, vol. 13009, Aug. 2021.
- [54] R. Williams, “The us now hosts more child sexual abuse material online than any other country,” <https://www.technologyreview.com/2022/04/26/1051282/the-us-now-hosts-more-child-sexual-abuse-material-online-than-any-other-country/>, 2022.
- [55] Y. Yang, Q. Liu, C. Brix, H. Zhang, and Y. Cao, “{CertPHash}: Towards certified perceptual hashing via robust training,” in *USENIX Security*, 2025.

Algorithm 6 Π_{authOPRF} : protocol realizing $\mathcal{F}_{\text{authOPRF}}$

Require: A group $\mathcal{G}$ with order q where the OM-gap Diffie Hellman problem is hard; a UF-CMA secure signature scheme $\Pi_{\text{Sig}} = (\text{SGEN}, \text{SSIGN}, \text{SVRFY})$; two random oracle $H_1 : \mathcal{U} \rightarrow \mathcal{G}$ and $H_2 : \{0, 1\}^{|\mathcal{U}|+|\mathcal{G}|} \rightarrow \{0, 1\}^\lambda$; collision-resistant hash function $h : \{0, 1\}^\lambda \rightarrow \mathcal{X}$

```

1: procedure Publish( $\mathcal{B}$ )
2:   for each  $A_k \in A_1, \dots, A_\eta$  do
3:     Sample key pair  $(pk_k, sk_k)$  for  $\Pi_{\text{Sig}}$ 
4:     Sample  $n$  random values  $r_{k,1}, \dots, r_{k,n} \leftarrow \mathbb{Z}_q$  and broadcast
       the values to all the other auditors.
5:     Wait until  $r_{j,1}, \dots, r_{j,n}$  are received from each auditor
        $A_j, j \in [1, \eta] \wedge k \neq j$ 
6:     for each  $b_i \in \mathcal{B}$  do
7:       Compute  $r_i \leftarrow \prod_{j \in [1, \eta]} r_{j,i}$ , and sets  $\mathcal{B}^{(\text{blind})} =$ 
          $\{\langle i, H_1(b_i)^{r_i} \rangle\}_{i \in [n]}$  and  $\mathcal{S} \leftarrow \{\langle i, b_i, r_i \rangle\}_{i \in [n]}$ 
8:       Sign  $\Sigma_k \leftarrow \text{SSIGN}_{sk_k}(h(\mathcal{B}^{(\text{blind})}))$ 
9:   The auditors publish  $\mathcal{B}^{(\text{blind})}$  and  $\{\Sigma_k\}_{k \in [1, \eta]}$ , and provide  $\mathcal{S}$  to
       SERVER

10: procedure Eval( $\mathcal{B}^{(\text{blind})}$ )
11:   After downloading  $\mathcal{B}^{(\text{blind})}$  and  $\{\Sigma_{sk_k}\}_{k \in [1, \eta]}$ , CLIENT aborts if
        $\text{SVRFY}_{pk_k}(\mathcal{B}^{(\text{blind})}, pk_k) \neq 1$  for some  $k \in [1, \eta]$ 
12:   CLIENT initializes a set  $\mathcal{S} \leftarrow \emptyset$  and samples a key  $\kappa_{\text{PRF}}^C \leftarrow \mathbb{Z}_q$ 
13:   For each  $\langle i, H_1(b_i)^{r_i} \rangle$  in  $\mathcal{B}^{(\text{blind})}$ , CLIENT adds
        $\langle i, (H_1(b_i)^{r_i})^{\kappa_{\text{PRF}}^C} \rangle$  to set  $\mathcal{S}'$ . Send  $\mathcal{S}'$  to SERVER
14:    $\triangleright$  Upon receiving  $\mathcal{S}'$  from CLIENT
15:   For every  $\langle i, (H_1(b_i)^{r_i})^{\kappa_{\text{PRF}}^C} \rangle \in \mathcal{S}'$ , let  $\langle i, b_i, r_i \rangle \in \mathcal{S}$ . SERVER
       computes  $((H_1(b_i)^{r_i})^{\kappa_{\text{PRF}}^C})^{r_i^{-1}} = H_1(b_i)^{\kappa_{\text{PRF}}^C}$  and outputs
        $\langle b_i, H_2(b_i, H_1(b_i)^{\kappa_{\text{PRF}}^C}) \rangle$ .

```

Appendix A.

Detectable Hash Function

The Apple PSI system introduced a *detectable hash function* (DHF). The following construction is detailed in their white paper and summarized in [40]. A DHF is an s-

Experiment $\text{EXPT}_{\Pi_{\text{Sig}}}^{\text{uf-cma}}(\mathbf{A}_{\text{ufcma}})$ $\langle pk, sk \rangle \leftarrow \text{GEN}(1^\lambda)$ $\langle S, \sigma \rangle \leftarrow \mathbf{A}_{\text{ufcma}}^{\text{SIGN}_{sk}(\cdot)}(pk)$ if $\text{VER}_{pk}(S, \sigma) = 1 \wedge S \setminus \phi \neq \emptyset$ where ϕ is the union of all the sets queried to $\text{SIGN}_{sk}(\cdot)$ then output 1 else output 0	Experiment $\text{EXPT}_{\Pi_{\text{Sig}}}^{\text{ind-sr}[b]}(\mathbf{A}_{\text{indsr}}^* = (\mathbf{A}_{\text{indsr}}[1], \mathbf{A}_{\text{indsr}}[2], \mathbf{A}_{\text{indsr}}[3]))$ $\langle pk, sk \rangle \leftarrow \text{GEN}(1^\lambda)$ $\langle S, \phi_1 \rangle \leftarrow \mathbf{A}_{\text{indsr}}[1]^{\text{SIGN}_{sk}(\cdot)}(pk)$ $\sigma_S \leftarrow \text{SIGN}_{sk}(S)$ $\langle y_0, y_1, \phi_2 \rangle \leftarrow \mathbf{A}_{\text{indsr}}[2]^{\text{SIGN}_{sk}(\cdot)}(\sigma, \phi_1)$ If $y_0 \in \phi \cup S \vee y_1 \in \phi \cup S$ where ϕ is the union of all the sets queried to $\text{SIGN}_{sk}(\cdot)$ then abort $b' \leftarrow \mathbf{A}_{\text{indsr}}[3](\text{RAND}_{pk}(\sigma \ominus_{pk} \{y_b\}), \phi_2)$ If $b' = b$ then output 1 else output 0
--	---

Figure 6: Security experiments for a randomizable set homomorphic signature

Algorithm 7 $\text{dsshare} = \langle \text{DSHARE}, \text{DCOMBINE} \rangle$

Require: A (ℓ_{ss}, t_{ss}) secret sharing scheme $\text{sshare} = \langle \text{SHARE}, \text{COMBINE} \rangle$; $(\ell_{ss} - t_{ss}, t_{ss} - 1)$ -detectable hash function $\text{DHF} : K_{dhf} \times \mathcal{D}_{dhf} \rightarrow \mathcal{R}_{dhf}$.

1: $\triangleright$ Secret sharing a value κ into ℓ_{ss} shares $\triangleleft$
2: **procedure** $\text{DSHARE}(\kappa)$
3: Secret-share κ into ℓ_{ss} shares $\{ss_1, \dots, ss_{\ell_{ss}}\} \leftarrow \text{SHARE}(\kappa)$
4: The dealer samples a key $k_{dhf} \leftarrow K_{dhf}$, and computes the set
5: $D \leftarrow \{\text{DHF}_{k_{dhf}}(r_i) : r_i \leftarrow \mathcal{D}_{dhf}, i \in [1, \ell_{ss}]\}$
6: **return** $SS \leftarrow \{ss_i, \text{DHF}_{k_{dhf}}(r_i) : i \in [1, \ell_{ss}]\}$
7: $\triangleright$ Reconstructing the detectable secret shares $\triangleleft$
8: **procedure** $\text{DCOMBINE}(SS)$ $\triangleleft$
9: $\triangleright$ Let $SS = \{\langle ss'_i, d'_i \rangle : i \in [1, \ell_{ss}]\}$
10: Compute the vector $\vec{z} \leftarrow \langle d'_i \rangle_{i \in [1, \ell_{ss}]}$
11: Compute $\text{DHF.DETECT}(\vec{z}) = \mathcal{I}$. If the algorithm fails, then abort.
 Otherwise, construct $S \leftarrow \{ss'_i : i \in \mathcal{I}\}$ and reconstruct the secret shares $\kappa \leftarrow \text{COMBINE}(S)$
12: **return** κ

detectable t -wise independent hash function $\text{DHF} : K_{dhf} \times \mathcal{D}_{dhf} \rightarrow \mathcal{R}_{dhf}$, such that s is an upper bound on the size of components, and $t + 1$ is the threshold number of matches before the server learns any data. A construction for an (s, t) -DHF is detailed below.

Let ℓ be a 64-bit prime, and $K_{dhf} \leftarrow \mathbb{F}_\ell^{s \times t}$, $\mathcal{D}_{dhf} \leftarrow \mathbb{F}_\ell$, $\mathcal{R}_{dhf} \leftarrow \mathbb{F}_\ell^{s+1}$. The key for the DHF k_{dhf} is a sequence of s polynomials $p_1, \dots, p_s \in \mathbb{F}_\ell[X]$ each of degree at most $t - 1$. Each row of the matrix k_{dhf} is one polynomial. Then,

$$\text{DHF}_{k_{dhf}}(x_0) \leftarrow (x_0, p_1(x_0), \dots, p_s(x_0)) \in \mathbb{F}_\ell^{s+1}$$

The output of $\text{DHF}_{k_{dhf}}(x_0)$ is a column vector in $\mathbb{F}_\ell^{s+1}$. A standard argument shows that the function DHF is weak t -wise independent: for random $k_{dhf} \leftarrow K_{dhf}$ and random $x_1, \dots, x_t \leftarrow \mathbb{F}_\ell$, the distribution $\text{DHF}_{k_{dhf}}(x_1), \dots, \text{DHF}_{k_{dhf}}(x_t)$ is uniform in $\mathcal{R}_{dhf}^t$.

Detection Algorithm: The detection algorithm is given a matrix of m columns, where each column is an element of $\mathcal{R}_{dhf} = \mathbb{F}_\ell^{s+1}$. It needs to detect those columns that are an output of DHF, assuming that there are at least $\max(t + 1, m - s)$ such columns. Recall that this means that there are at least $t + 1$ DHF columns and at most s random columns.

This problem is related to decoding the interleaved Reed-Solomon code under random noise, and can be solved using a technique described by Coppersmith and Sudan [17]

1) Expand each of the m given column vectors $\mathbf{r} \leftarrow (x_0, r_1, \dots, r_s) \in \mathbb{F}_\ell^{s+1}$ into a column vector $\mathbf{r}' \in \mathbb{F}_\ell^{s+t}$

$$\mathbf{r}' \leftarrow (1, x_0, x_0^2, \dots, x_0^{t-1}, r_1, \dots, r_s) \in \mathbb{F}_\ell^{s+t}$$

2) Let $M \in \mathbb{F}_\ell^{(s+t) \times m}$ be the resulting matrix after expansion. Every row of M contains at least $t + 1$ evaluations of some polynomial f of degree at most $t - 1$. Hence, the $t + 1$ correct DHF columns of M must be linearly independent.

3) If $\dim(\text{kernel}(M)) = 0$ then output $\perp$. Otherwise:

- Let $\mathbf{w} \in \mathbb{F}_\ell^m$ be a vector such that $M \cdot \mathbf{w} = 0$.
- Let $Z \subseteq [1, m]$ be the set of indices where $\mathbf{w} \neq 0$.
- Let $M[Z] \in \mathbb{F}_\ell^{(s+t) \times |Z|}$ be the sub-matrix of M restricted to the columns in Z .
- Output the set of indices $\hat{Z} \subseteq [m]$ where $i \in \hat{Z}$ iff. column number i of M is in the linear span of the columns in $M[Z]$. Note that $\hat{Z}$ is a super set of Z .

The correctness of the algorithm is implied by the construction in [17]. The algorithm has $\mathcal{O}((s+t)m^2) \in \mathcal{O}(s^3)$ cost since it requires a matrix decomposition over M .

Appendix B.

Proofs for Signature Scheme in Sec. 4.2

Definition 3 (Strong QR-RSA problem). Assume that $N = pq$ where p and q are random, safe prime numbers, and QR_N is the quadratic residue subgroup of $\mathbb{Z}_N^*$. Then, for $y \leftarrow QR_N$,

$$\begin{aligned} \text{Adv}_N^{\text{QR-RSA}}(\mathbf{A}_{\text{QR-RSA}}) &= \\ \mathbb{P} \left(x \in QR_N \wedge e \geq 2 \wedge \begin{array}{c} x^e \equiv y \pmod{N} \end{array} \middle| (x, e) \leftarrow \mathbf{A}_{\text{QR-RSA}}(N, y) \right) \\ \text{Adv}_N^{\text{QR-RSA}}(t) &= \max_{\mathbf{A}_{\text{QR-RSA}}} \text{Adv}_N^{\text{QR-RSA}}(\mathbf{A}_{\text{QR-RSA}}) \end{aligned}$$

where the probability is over the choice of N and y and random choices by $\mathbf{A}_{\text{QR-RSA}}$, and the maximum is taken over all adversary algorithms $\mathbf{A}_{\text{QR-RSA}}$ that run in total time t .

$\mathcal{F}_{\text{dsign}}$: Distributed Signing Functionality

Parameters: A randomizable homomorphic signature scheme $\Pi_{\text{RH-Sig}} = \langle \text{GEN}, \text{SIGN}, \text{VRFY}, \text{RAND}, \text{AUTH}, \text{BLIND}, \text{UNBLIND} \rangle$. η auditors $A_1, \dots, A_\eta$.

Inputs: The auditors have a common list of item $\mathcal{B} \subseteq \mathcal{U}$. Auditor A_k has the share of a private key $sk_k^{(d)}$. The public key is $pk^{(d)}$

Functionality:

- Upon receiving $(\text{DSIGN}, A_k, \mathcal{B}, sk_k^{(d)})$ from A_k , store $\mathcal{B}, sk_k$. After receiving the DSIGN message from all auditors, check that all the messages have the same input $\mathcal{B}$. If so, output $\text{SIGN}_{sk^{(d)}}(\mathcal{B})$, otherwise abort.
- Upon receiving $(\text{DBLIND}, A_k, \sigma_{\mathcal{B}}, r, sk_k^{(d)})$ from A_k , store $\sigma_{\mathcal{B}}, sk_k$. After receiving the DBLIND message from all auditors, check that all the messages have the same input $\sigma_{\mathcal{B}}$ and $r \in [1, N^2]$. If so, output $\text{BLIND}_{sk^{(d)}}(\sigma_{\mathcal{B}}, r)$, otherwise abort.
- Upon receiving $(\text{DADD}, A_k, \mathcal{B}, \mathcal{S}, sk_k^{(d)})$ from A_k , store $\mathcal{B}, \mathcal{S}, sk_k^{(d)}$. After receiving the DADD from all auditors, check that all the messages have the same inputs $\mathcal{B}$ and $\mathcal{S}$. If so, output $\text{SIGN}_{sk^{(d)}}(\mathcal{B} \cup \mathcal{S})$, otherwise abort.
- Upon receiving $(\text{DDEL}, A_k, \mathcal{B}, \mathcal{S}, sk_k^{(d)})$ from A_k , store $\mathcal{B}, \mathcal{S}, sk_k^{(d)}$. After receiving the DDEL from all auditors, check that all the messages have the same inputs $\mathcal{B}$ and $\mathcal{S}$. If so, output $\text{SIGN}_{sk^{(d)}}(\mathcal{B} \setminus \mathcal{S})$, otherwise abort.

Π_{dsign} : Distributed Signing Protocol Realizing $\mathcal{F}_{\text{dsign}}$

Setup (one-time):

- Auditors generate a shared modulus N using the protocols of Algesheimer et al. [2], Ong and Kubiawicz [39]. Auditor A_k receives additive shares of the Sofie-Germaine primes $\langle p' \rangle_k, \langle q' \rangle_k$. Using Given the shares, the auditors compute additive shares of N , and additive shares of the order of the quadratic residue subgroup $\phi(QR_N) = p'q'$, using the BGW protocol [5]
- Auditors select a generator g :
 - 1) Sample a random value $g \in [2, N-1]$ using a distributed randomness beacon [13]
 - 2) Verify (locally) that the Jacobi symbol $(\frac{g}{N}) \neq -1$. If so g is a quadratic non-residue. Go to step (1) and repeat.
 - 3) If $(\frac{g}{N}) = 1$, run a BGW instance with $g^{\langle p' \rangle_k}$ and $g^{\langle q' \rangle_k}$ as inputs and check if $\prod_{k=1}^{\eta} g^{\langle p' \rangle_k} \bmod N \neq 1 \wedge \prod_{k=1}^{\eta} g^{\langle q' \rangle_k} \bmod N \neq 1$. If the check is successful, return g as the generator. Otherwise, go to step (1) and repeat.

Realizing DSIGN

- 1) Compute $U \leftarrow \prod_{b_i \in \mathcal{B}} H_{\mathcal{P}}(b_i)$
 - a) Using the protocol of Catalano et al. [12, Fig 3], compute additive shares of $U^{-1} \bmod \phi(QR_N)$. Auditor A_k gets share $\langle x \rangle_k$.
 - b) Compute $\sigma_{\mathcal{B}} \leftarrow g^{\left(\sum_{k \in [1, \eta]} \langle x_i \rangle_k \right)}$ using a ring network: Auditor A_k waits to receive $\sigma' \leftarrow g^{\left(\sum_{m \in [1, k-1]} \langle x_i \rangle_m \right)}$ from A_{k-1} . Upon receiving σ' , it forwards $\sigma' g^{\langle x_i \rangle_k} = g^{\left(\sum_{m \in [1, k]} \langle x_i \rangle_m \right)}$ to A_{k+1} . Auditor A_k finally obtains and broadcasts $\sigma_{\mathcal{B}} = \text{SIGN}_{sk^{(d)}}(\mathcal{B})$

Realizing DADD

Use the mechanism described above to sign $\mathcal{S}$, $\sigma_{\mathcal{S}} = \text{SIGN}_{sk^{(d)}}(\mathcal{S})$. Then, compute $\text{SIGN}_{sk^{(d)}}(\mathcal{B} \cup \mathcal{S}) = \sigma_{\mathcal{B}} \odot_{pk} \sigma_{\mathcal{S}}$

Realizing DDEL

Compute $\text{SIGN}_{sk^{(d)}}(\mathcal{B} \setminus \{b\}) = \sigma_{\mathcal{B}}^{H_{\mathcal{P}}(b)} \bmod N$

Realizing DBLIND:

Auditors sample a random number $\beta \leftarrow_{\$} [1, N^2]$ using a distributed randomness beacon [13] and using Catalano et al. [12, Fig 3] compute shares of $\beta^{-1} \bmod \phi(QR_N)$. Then, using the protocol above compute and output $\langle \sigma_{\mathcal{B}}^{\frac{1}{\beta}}, \beta \rangle$.

Figure 7: Protocol for distributed signing realizing $\mathcal{F}_{\text{dsign}}$

Theorem B.1. Assuming that strong QR-RSA problem is hard to solve in group QR_N .

$$\text{Adv}_{\Pi_{\text{RH-Sig}}}^{\text{uf-cma}}(t, q_H) \leq \frac{q_H^2 \log N}{N} + q_H \text{Adv}_N^{\text{QR-RSA}}(t)$$

Here q_H is the number of distinct queries to $\text{SIGN}_{sk}(\cdot)$

Proof. We will show that if there is an adversary A_{ufcma} that can forge a signature in time t , then there is an adversary $A_{\text{QR-RSA}}$ that solves the strong QR-RSA problem with probability $\text{Adv}_N^{\text{QR-RSA}}(t)$ in time t . The reduction works as follows. Consider that $A_{\text{QR-RSA}}$ simulates A_{ufcma} 's queries to the signing oracle, and the random oracle $H_{\mathcal{P}}$. Let $\mathcal{S}_1, \dots, \mathcal{S}_{n_s}$ be all the sets that are queried by A_{ufcma} to the signing oracle. Assume that q_H is the total number of queries to the random oracle that includes queries by

the signing oracle when responding to A_{ufcma} , and queries directly by A_{ufcma} . Now assume that A_{ufcma} forges a signature on some value y that is not in any of the sets $\mathcal{S}_1, \dots, \mathcal{S}_{n_s}$. This happens under two conditions.

- 1) There is some $y' \in \bigcup_{i \in [i]} \mathcal{S}_i$ such that $H_{\mathcal{P}}(y') = H_{\mathcal{P}}(y)$.
- 2) For all $y' \in \mathcal{S}_1, \dots, \mathcal{S}_{n_s}$ $H_{\mathcal{P}}(y') \neq H_{\mathcal{P}}(y)$.

The first case happens with probability at most $\frac{q_H^2 \log N}{N}$ because $H_{\mathcal{P}}$ maps to prime numbers in $[1, N]$ and there are roughly $\frac{N}{\log N}$ prime numbers less than N .

For the second case, consider the following simulation

- 1) $A_{\text{QR-RSA}}$ selects q_H random prime numbers less than N , $p_1, \dots, p_{q_H}$ and computes $P \leftarrow \prod_{i \in [1, q_H]} p_i$. Then, it computes $z \leftarrow y^P \bmod N$. $A_{\text{QR-RSA}}$ publishes z as the genotor in the public key.

- 2) AQR-RSA selects a random index $j \in [1, q_H]$ and programs the random oracle $H_{\mathcal{P}}$ such for the i -th oracle query for all $i \neq j$, the random oracle returns p_i . For the j -th query, the random oracle returns a random prime number $e \notin p_1, \dots, p_{q_H}$
- 3) AQR-RSA answers the signing oracle queries from A_{ufcma} for the sets $\mathcal{S}_1, \dots, \mathcal{S}_{n_s}$. Specifically, given some set $\mathcal{S}$, AQR-RSA signs $\mathcal{S}$ by computing $P' \leftarrow \{p_i : p_i \in p_1, \dots, p_{q_H}, \forall x \in \mathcal{S}, H_{\mathcal{P}}(x) \neq p_i\}$ and returning $X \leftarrow y^{P'} = z^{\left(\frac{1}{\prod_{x \in \mathcal{S}} H_{\mathcal{P}}(x)}\right)}$
- 4) If A_{ufcma} forges a signature on $\mathcal{S}'$ such that there is some $x \in \mathcal{S}'$ but $x \notin \mathcal{S}_1 \cup \dots \cup \mathcal{S}_{n_s}$ and $H'_{\mathcal{P}}(x) = e$, then AQR-RSA learns some value v such that $v^e = y^P \bmod N$. Since, e is co-prime to P , we can find the values of a, b using the extended Euclid algorithm such that $ae + bP = 1$. Then, we have $(v^b y^a)^e = y$. AQR-RSA outputs $v^b y^a$ as the e -th root of y

Note that in the real experiment when A_{ufcma} queries the set $\mathcal{S}$, its view includes g^{1/U_i} where $U_i \leftarrow \prod_{x \in \mathcal{S}_i} H_{\mathcal{P}}(x)$ and g is a generator of QR_N . In the simulation, A_{ufcma} 's view includes $z^{\left(\frac{1}{\prod_{x \in \mathcal{S}} H_{\mathcal{P}}(x)}\right)}$. Since y is selected randomly from QR_N , the probability that it is not a generator is negligible in λ . Consequently, $z \leftarrow y^P$ is also a generator with overwhelming probability, i.e., as long as P is co-prime to p' and q' . Thus, the signature in the simulation is valid and indistinguishable to A_{ufcma} . Finally, AQR-RSA solves the strong QR-RSA problem using the simulation when it programs the random oracle such that $H_{\mathcal{P}}(x) = e$, this happens with probability at least $1/q_H$. If A_{ufcma} is successful in the forging a signature in the simulation with probability $\text{Adv}_{\Pi_{\text{RH-Sig}}}^{\text{uf-cma}}(t, q_H)$, then AQR-RSA solves the strong QR-RSA problem also with probability $\geq \text{Adv}_N^{\text{QR-RSA}}(t)/q_H$. $\square$

Theorem B.2. Assuming the strong QR-RSA problem is hard to solve in QR_N where N is the product of safe primes.

$$\text{Adv}_{\Pi_{\text{RH-Sig}}}^{\text{ind-sr}}(t, q_H, q'_H) \leq \frac{q'_H}{|QR_N|} + q_H \text{Adv}_N^{\text{QR-RSA}}(t)$$

Here q_H is the number of distinct queries to the random oracle $H_{\mathcal{P}}$ and q'_H is the number of distinct queries to the random oracle H .

Proof. Consider that $A_{\text{indsr}}[3]$ gets a challenge $\langle g^{\left(\frac{rH_{\mathcal{P}}(x_b)}{U}\right)} \bmod N, H(g^r \bmod N) \rangle$ where $g^{\frac{1}{U}} \bmod N$ is a signature on the set $\mathcal{S}$. Since H is a random oracle and $g^r \bmod N$ is uniformly distributed in QR_N [18], $A_{\text{indsr}}[3]$ gets $g^r \bmod N$ from $H(g^r \bmod N)$ only if it is able to compute it from the challenge, or if it has correctly guessed it in q'_H queries. The latter happens with probability at most $q'_H/|QR_N|$.

If $A_{\text{indsr}}[3]$ can distinguish between $g^{\left(\frac{rH_{\mathcal{P}}(x_0)}{U}\right)} \bmod N$ and $g^{\left(\frac{rH_{\mathcal{P}}(x_1)}{U}\right)} \bmod N$ then it can also distinguish between $g^{rH_{\mathcal{P}}(x_0)} \bmod N$ and $g^{rH_{\mathcal{P}}(x_1)} \bmod N$ because $A_{\text{indsr}}[3]$

knows U . Let $r = ap'q' + b$. It is known that b is uniformly distributed in $\mathbb{Z}_{p'q'}$ when $r \leftarrow_{\$} [1, N^2]$ [18]. Thus, $rH_{\mathcal{P}}(x_b) \bmod p'q'$ is uniformly distributed in $\mathbb{Z}_{p'q'}$ and $g^{rH_{\mathcal{P}}(x_0)} \bmod N$ is statistically indistinguishable from $g^{rH_{\mathcal{P}}(x_1)} \bmod N$ since g is a generator of QR_N .

Finally, if $A_{\text{indsr}}[3]$ queries the random oracle for $g^r \bmod N$ without guessing, then it has computed $g^{rH_{\mathcal{P}}(x_b)H_{\mathcal{P}}(x_b)^{-1} \bmod p'q'} \bmod N$. In this case, AQR-RSA can solve the strong QR-RSA problem with the help of A_{indsr}^* . Consider the following simulation where AQR-RSA programs the random oracles $H, H'_{\mathcal{P}}$.

- 1) AQR-RSA selects q_H random prime numbers less than N , $p_1, \dots, p_{q_H}$ and computes $P \leftarrow \prod_{i \in [1, q_H]} p_i$. Then, it computes $z \leftarrow y^P \bmod N$. AQR-RSA publishes z as the generator in the public key.
- 2) AQR-RSA selects a random index $j \in [1, q_H]$ and programs the random oracle $H_{\mathcal{P}}$ such for the i -th oracle query for all $i \neq j$, the random oracle returns p_i . For the j -th query, the random oracle returns a random prime number $e \notin p_1, \dots, p_{q_H}$
- 3) AQR-RSA answers the signing oracle queries from $A_{\text{indsr}}[1]$ and $A_{\text{indsr}}[2]$ for the sets $\mathcal{S}_1, \dots, \mathcal{S}_{n_s}$. Specifically, upon receiving $\mathcal{S}$ from $A_{\text{indsr}}[1]$, AQR-RSA returns $z^{\left(\frac{1}{\prod_{x \in \mathcal{S}} H_{\mathcal{P}}(x)}\right)} \bmod N$ (see the proof of Theorem B.1 to see how this is computed). If $H_{\mathcal{P}}(x) = e$ for some $x \in \mathcal{S}$, then the simulation aborts.
- 4) Upon receiving x_0, x_1 from $A_{\text{indsr}}[2]$, AQR-RSA checks that these elements are not in any of the sets queried by $A_{\text{indsr}}[1]$ and $A_{\text{indsr}}[2]$. Note that the values of $H_{\mathcal{P}}(x_0)$ and $H_{\mathcal{P}}(x_1)$ are in $p_1, \dots, p_{j-1}, e, p_{j+1}, \dots, p_{q_H}$.
- 5) AQR-RSA computes $P' \leftarrow \{p_i : p_i \in p_1, \dots, p_{q_H}, \forall x \in \mathcal{S}, H_{\mathcal{P}}(x) \neq p_i\}$.
- 6) AQR-RSA computes $X \leftarrow y^{r'P'} = z^{\left(\frac{r'}{\prod_{x \in \mathcal{S}} H_{\mathcal{P}}(x)}\right)}$ where $r' \leftarrow_{\$} [1, N^2]$ and sends the challenge $\langle X, r'' \rangle$ to $A_{\text{indsr}}[3]$ where $r'' \leftarrow_{\$} \{0, 1\}^\lambda$.
- 7) If $A_{\text{indsr}}[3]$ queries H on some value v such that $v^e = z^{r'} = y^{r'P'}$, then AQR-RSA computes the values of a, b using the extended Euclid algorithm such that $ae + br'P' = 1$. Note that e is co-prime to $r'P'$ with overwhelming probability. Then, we have $(v^b(y)^a)^e = y$. AQR-RSA outputs $v^b(y)^a$ as the e -th root of y

Observe that $z^{\left(\frac{1}{\prod_{x \in \mathcal{S}} H_{\mathcal{P}}(x)}\right)} \bmod N$ is a valid and an indistinguishable signature (see the proof of Theorem B.1 for

details). Also, as discussed before $z^{\left(\frac{r'}{\prod_{x \in \mathcal{S}} H_{\mathcal{P}}(x)}\right)} \bmod N$ is a uniformly sampled element in QR_N and is indistinguishable from $z^{\left(\frac{r x_b}{\prod_{x \in \mathcal{S}} H_{\mathcal{P}}(x)}\right)} \bmod N$ because $z^{\left(\frac{x_b}{\prod_{x \in \mathcal{S}} H_{\mathcal{P}}(x)}\right)} \bmod N$ is a generator of QR_N with overwhelming probability. Thus, the simulation succeeds and AQR-RSA wins when it programs either of the challenges x_0 and x_1 as $H_{\mathcal{P}}(x_0) = e$ or $H_{\mathcal{P}}(x_1) = e$, which happens with probability $\geq 1/q_H$. $\square$

Appendix C.

Proofs of functionality

Theorem C.1. *Assuming that there are protocols securely realizing $\mathcal{F}_{\text{sign}}$ and $\mathcal{F}_{\text{authOPRF}}$, Π_{EM} securely realizes $\mathcal{F}_{\text{Mod}}$*

Proof. We show that there is a polynomial time simulator that can simulate CLIENT's and SERVER's views in Π_{EM}

Simulating CLIENT's view: The client is semi-honest in our model. The simulator implements the random oracles $H_{\mathcal{P}}$ and H for the signature based ACB, and the simulation proceeds as follows:

- 1) During an upload, the simulator gets ID from the message sent by CLIENT to SERVER with the file.
- 2) When CLIENT queries the random oracle $H_{\mathcal{P}}$ on some $y \in \mathcal{U}$, the simulator returns a random value $r \leftarrow \mathcal{P}$ as the output of $H_{\mathcal{P}}(y)$ and stores $\langle y, r \rangle$
- 3) When CLIENT queries the random oracle $H_{\mathcal{P}}$ on some $y' \in QR_N$, the simulator returns a random value $w \leftarrow \{0, 1\}^\lambda$ as the output of $H(y')$ and stores $\langle y', w \rangle$
- 4) The simulator sends $\langle \text{UPLOADFILE}, \text{ID}, y \rangle$ to $\mathcal{F}_{\text{ExactMod}}$.
- 5) If CLIENT's output tape has $\langle \text{ID}, \text{detected} = \text{TRUE} \rangle$, the simulator simulates the the fine-grained check by sending a random set of indices $\hat{\mathcal{I}} \leftarrow \mathcal{I}$ to CLIENT.
- 6) If CLIENT's output tape has $\langle \text{ID}, \text{detected} = \text{FALSE} \rangle$, the simulator simulates a false positive by sending a set of indices $\hat{\mathcal{I}} \leftarrow \mathcal{I}$ to CLIENT with probability ϵ_{bf} .

The simulation is indistinguishable from the real protocol execution because CLIENT's messages in the simulation include the r, w when detected = FALSE. Due to the random oracle assumption, these messages are statistically indistinguishable from the messages received in the protocol from the random oracle. When detected = TRUE, CLIENT receives a random set of indices from the simulation. This is indistinguishable from the actual set of indices $\mathcal{I}$ received by CLIENT in the real protocol execution. This is because the items in the Bloom filter bf_m sent by CLIENT are the items $b \in \mathcal{B}$ such that $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(b)) \equiv k \bmod \zeta$. Since $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(b))$ is a pseudorandom value, the distribution of b in bf_m is uniformly random. Thus, their corresponding mappings to buckets are also uniformly random.

The simulator simulates false positives by requesting a fine-grained check with probability ϵ_{bf} . A fine-grained check is necessary, in case of a false positive, when there is a collision in the Bloom filter bf_m between $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(b))$ and $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(y))$ for some $b \in \mathcal{B}$. Again, since these are pseudorandom values, the probability of collision does not depend on CLIENT's input y and b . Therefore, the simulation only needs ϵ_{bf} to simulate a false positive without information about b .

Simulating SERVER's view: SERVER is malicious in our model. The simulation proceeds as follows

- **Preprocessing**

- 1) During preprocessing, the simulator obtains $\langle b_i, z_i \rangle$ from $\mathcal{F}_{\text{authOPRF}}$ and adds it to a list L for each $b_i \in \mathcal{B}$. The simulator sends $\{z_1, \dots, z_n\}$ to SERVER.
- 2) The simulator programs the random oracle $H_{\mathcal{P}}(b_i) \leftarrow \mathcal{P}$ for each $b_i \in \mathcal{B}$
- 3) The simulator gets the encrypted Bloom filters from SERVER's messages to CLIENT $\{\text{ENC}_{\kappa}(bf_1), \dots, \text{ENC}_{\kappa}(bf_{\tau})\}$ after the preprocessing phase

- **Upload**

- The simulator gets the set of SERVER's outputs from a polynomial number of uploads that have two types. Type 1: $\langle \text{detected} = \text{TRUE}, \text{ID}, \text{DATA} \rangle$ and Type 2: $\langle \text{detected} = \text{FALSE}, \emptyset \rangle$
- 1) For every output of type 1 $\langle \text{detected} = \text{TRUE}, \text{ID}, \text{DATA} \rangle$
 - a) The simulator computes $y \leftarrow \text{embed}(\text{DATA})$ and sends the message $(\text{CHECKFILE}, \text{ID}, y)$ to $\mathcal{F}_{\text{Mod}}$. If $\mathcal{F}_{\text{Mod}}$ aborts, then the simulation aborts.
 - b) The simulator sends z and $\text{ENC}_{\kappa}(bf_k)$ to SERVER where there is an entry $\langle y, z \rangle$ in L where $z \equiv k \bmod \zeta$
 - c) If SERVER aborts, then the simulator aborts. Otherwise if SERVER sends $\mathcal{I}$, the simulator computes and sends to SERVER $\sigma_{\mathcal{B}^{(m)}}^{r_m \cdot H_{\mathcal{P}}(y)}$, where $r_m \leftarrow [1, N^2]$, and $R_m \leftarrow \{0, 1\}^\lambda$ for $m \in \mathcal{I}$. The simulator stores $\langle r_m, R_m \rangle$
 - d) The simulator answers queries to the random oracle H for SERVER. If SERVER sends a query for $g^{r_m} \bmod N$ for some r_m used in the previous step, then the simulator responds to the query by outputting R_m . Otherwise, the simulator responds with a random element in $\{0, 1\}^\lambda$ and stores the output
 - e) The simulator outputs whatever SERVER outputs
 - 2) For every output of Type 2 $\langle \text{detected} = \text{FALSE}, \emptyset \rangle$
 - a) The simulator samples a random value $\hat{z} \leftarrow \{0, 1\}^\lambda$ and sends to server $\hat{z}$ and $\text{ENC}_{\kappa}(bf_k)$ where $\hat{z} \equiv k \bmod \zeta$
 - b) If SERVER aborts, then the simulator abort. Otherwise, receive $\mathcal{I}$ from the adversary.
 - c) The simulator selects some $\hat{y} \in \mathcal{U} \setminus \mathcal{B}$, and programs the random oracle such that $H_{\mathcal{P}}(\hat{y}) \leftarrow \mathcal{P}$
 - d) For each $m \in \mathcal{I}$, the simulator computes and sends to SERVER $\sigma_{\mathcal{B}^{(m)}}^{r_m \cdot H_{\mathcal{P}}(\hat{y})}$, where $r_m \leftarrow [1, N^2]$ and $R_m \leftarrow \{0, 1\}^\lambda$ for $m \in \mathcal{I}$. The simulator stores $\langle r_m, R_m \rangle$
 - e) The simulator responds to queries to H from SERVER. If there is a query for g^{r_m} for some r_m selected in the previous step, then the simulator aborts. Otherwise, the simulator responds with a random element $\{0, 1\}^\lambda$ and stores the output
 - f) The simulator outputs whatever SERVER outputs

The simulation is indistinguishable from the real execution. During processing, SERVER's messages includes outputs of $\mathcal{F}_{\text{authOPRF}} \{\text{PRF}_{\kappa_{\text{PRF}}}^C(b_1), \dots, \text{PRF}_{\kappa_{\text{PRF}}}^C(b_n)\}$

while in the simulation, the SERVER's messages includes $\{z_1, \dots, z_n\}$. These are indistinguishable assuming that the outputs of PRF are indistinguishable from random outputs in $\{0, 1\}^\lambda$.

Now, consider the following two cases during uploads

Type 1 output: In this case, SERVER's messages includes $\text{PRF}_{\kappa_{\text{PRF}}}^C(y)$, $\text{ENC}_{\kappa}(bf_k)$, $\sigma_{\mathcal{B}(m)}^{r_m H^{\mathcal{P}}(y)}$, $H(g^{r_m})$ in the real execution for all $m \in \mathcal{I}$. In the ideal world, SERVER's messages includes z , $\text{ENC}_{\kappa}(bf_k)$, $\sigma_{\mathcal{B}(m)}^{r_m H^{\mathcal{P}}(y)}$, R_m for all $m \in \mathcal{I}$. Assuming that PRF produces outputs indistinguishable from random $\text{PRF}_{\kappa_{\text{PRF}}}^C(y)$ and z are indistinguishable to the adversary. Also, assuming that H is a random oracle $H(g^{r_m})$ is indistinguishable from R_m unless the adversary has queried the random oracle on H on r_m before which happens with negligible probability for a fresh randomness r_m used in every query, since g^{r_m} is a random element in QR_N [18].

Type 2 output: Observe that in the type 2 case, there are two possibilities: i) the check terminates with the fast path, or ii) the check proceeds to a fine grained. Since the messages observed by the adversary in the second case subsumes the messages in the first case, we argue indistinguishability for this case. Later, we show that the distribution of these cases occurring does not break indistinguishability.

When the protocol performs a fine-grained check, SERVER's messages includes $\text{PRF}_{\kappa_{\text{PRF}}}^C(y)$, $\text{ENC}_{\kappa}(bf_k)$, $\sigma_{\text{Part}_m}^{r_m H^{\mathcal{P}}(y)}$, $H(g^{r_m})$ such that $y \notin \mathcal{B}$. In the simulation, SERVER's messages include $\hat{z}$, $\text{ENC}_{\kappa}(bf_k)$, $\sigma_{\text{Part}_m}^{r_m H^{\mathcal{P}}(\hat{y})}$, R_m for all $m \in \mathcal{I}$. Observe that when $y \notin \mathcal{B}$, the adversary does not know $\text{PRF}_{\kappa_{\text{PRF}}}^C(y)$ in the real execution from the preprocessing phase. Thus, assuming that the PRF produces indistinguishable outputs from random $\text{PRF}_{\kappa_{\text{PRF}}}^C(y)$ is indistinguishable from $\hat{z}$. Also, due to H being a random oracle $H(g^{r_m})$ is indistinguishable from R_m as discussed above. Finally, if the adversary can distinguish between $\sigma_{\text{Part}_m}^{r_m H^{\mathcal{P}}(y)}$ and $\sigma_{\text{Part}_m}^{r_m H^{\mathcal{P}}(\hat{y})}$ when $y, \hat{y} \notin \mathcal{B}$, then it violates the IND-SR property of the signature scheme, which happens with negligible probability as per Theorem B.2.

The probability that any of the type 2 cases terminates with the fast path depends on the false positives in the Bloom filter, which in turn depends on potential collisions between $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(b))$ and $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(y))$ for some $b \in \text{Bin}_k$. In the simulation, a false positive occurs when there is a collision between $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(b))$ and $\text{PRP}_{\kappa_{\text{PRP}}}^S(z)$. Since $\text{PRF}_{\kappa_{\text{PRF}}}^C(b)$ and z are indistinguishable due to the PRF, the probability of a false positive in the simulation is distributed indistinguishably to the probability of a false positive in the real execution. $\square$

Theorem C.2. *Assuming that there is an IND-CPA secure encryption scheme, a PRG producing outputs indistinguishable from random and a detectable secret-sharing scheme, Π_{FM} securely realizes $\mathcal{F}_{\text{Mod}}$*

Proof. We show that there is a polynomial time simulator that can simulate CLIENT's and SERVER's views in Π_{FM}

Simulating client's view The ACB is an encrypted cuckoo hash table, which protects confidentiality assuming that the

encryption scheme is IND-CPA secure. The client does not obtain any outputs from the upload phase.

Simulating the server's view The SERVER has two types of output from an upload. Type 1: $\langle \text{detected} = \text{TRUE}, \text{ID}, \phi \rangle$ and Type 2: $\langle \text{detected} = \text{FALSE}, \emptyset \rangle$

For type 1 output, the simulator computes $ss_1^*, \dots, ss_{2\gamma}^* \leftarrow \text{DSHARE}(0)$, and 2γ keys for a PRG $\kappa_1, \dots, \kappa_{2\gamma} \leftarrow \mathcal{K}_{\text{PRG}}$. The simulator also samples set of indices $\mathcal{I} \leftarrow [1, 2\gamma]$, such that $|\mathcal{I}| = \phi$.

Then, it computes for $j \in [1, 2\ell]$

$$\hat{u}_j \leftarrow \text{Enc}_{pk_e}(\kappa_j)$$

For $i \in \mathcal{I}$, the simulator computes

$$\hat{u}_i^* \leftarrow \text{PRG}(\kappa_i) \oplus ss_i^*$$

For $i \notin \mathcal{I}$, the simulator computes

$$\hat{u}_i^* \leftarrow r_i \oplus ss_i^*; \quad r_i \leftarrow \{0, 1\}^{|ss_i^*|}$$

The simulator outputs to the server $\{\hat{u}_j, \hat{u}_j^*\}_{j \in [1, 2\ell]}$

For type 2 output, the simulator calculates $\hat{u}_j$ for $j \in [1, 2\ell]$ as above, but sets for all $j \in [1, 2\ell]$

$$\hat{u}_j^* \leftarrow r_j; \quad r_j \leftarrow \{0, 1\}^{|ss_i^*|}$$

In the real execution, for type 1 outputs, the server is able to decrypt ϕ PRG keys, and thus obtain *numMatchingProj* secret-shares. The rest of the secret shares cannot be decrypted by the server unless it is able to compute the PRG output without the key. The simulation is indistinguishable if the PRG produces outputs that are indistinguishable from random. A similar argument holds for the Type 2 outputs, where none of the secret-shares can be decrypted by the server in the simulation. Assuming that the DSS protocol hides the number of correct secret shares in the Type 2 outputs, the simulation is indistinguishable. $\square$